

A COMPLETE BEGINNER'S LEARNING HANDBOOK

EDITION 1.0 • WEEK 01

Python Programming Foundations

*From your first line of code to object-oriented programming
— a guided journey for absolute beginners.*

This handbook accompanies you through every concept a new programmer needs in Week 1: setting up Python, writing your first programs, mastering control flow, organising data, building functions, handling files, and writing your first classes. Each chapter ends with a real, build-along project so the theory becomes muscle memory.

```
>>> print("Hello, Python!")
Hello, Python!
>>> for i in range(3): print(f"Day {i+1}: learning")
Day 1: learning
Day 2: learning
Day 3: learning
```

CURRICULUM DESIGN TEAM

Python Programming Foundations • Week 1
Beginner Track • Classroom & Self-Study Edition

HANDBOOK

01 / 05

CONTENTS

What you'll learn

01	Orientation & Welcome to Python	03
	Why programming, why Python, what to expect this week	
02	Setting Up Your Python Environment	06
	Installing Python, VS Code and Google Colab	
03	Variables, Data Types & Input/Output	12
	Storing values, types, print() and input()	
04	Operators in Python	19
	Arithmetic, comparison, logical, assignment	
05	Project 1 — Calculator Program	24
	Your first complete Python application	
06	Control Flow — Making Decisions	28
	if, elif, else, indentation, flowcharts	
07	Control Flow — Loops & Iteration	34
	for, while, break, continue	
08	Project 2 — Number Guessing Game	40
	Bringing decisions and loops together	
09	Strings — Python's Text Data Structure	44
	Indexing, slicing, methods, immutability	
10	Lists, Tuples, Sets & Dictionaries	50
	The four core collections compared	
11	Project 3 — Student Management Program	58
	A CLI app using lists of dictionaries	
12	Functions & Modular Programming	62
	def, parameters, return, scope	

13	Modules, Exceptions & File Handling	68
	import, try/except, text & CSV files	
14	Project 4 — Expense Tracker Application	75
	Functions + files + exceptions in one app	
15	Introduction to Object-Oriented Programming	80
	Classes, objects, __init__, self	
16	Project 5 — Library Management System (Mini Project)	85
	Capstone applying the entire week	
17	Weekly Revision, Quiz & What's Next	91
	Recap, 15-question quiz, answer key	

Orientation & Welcome to Python

Welcome to your first week of Python programming. Whether you have never written a single line of code in your life, or you have tinkered with a few tutorials and ended up more confused than when you started, this handbook is written for you. Take a breath. You are exactly where you should be.

Before we install anything, before we type a single keyword, let us spend a few minutes understanding what programming actually is, why Python is the language we have chosen for you, and what your week is going to look like. A clear mental map of the journey makes every step that follows easier.

1.1 What is programming, really?

A computer, on its own, is a remarkably fast but completely obedient machine. It will do exactly what you tell it to do — nothing more, nothing less. The catch is that it does not understand human languages like English, Hindi or Tamil. It only understands precise, unambiguous instructions.

Programming is the act of writing those instructions in a language the computer can follow. Think of it as writing a recipe. A recipe tells the cook, step by step, what to do: *boil the water, add the rice, cover for fifteen minutes, serve*. A program tells the computer, step by step, what to do: *take this input, do this calculation, show this output*. The cook and the computer share one trait — both will fail if your instructions are vague.

Once you can write clear instructions for a computer, you can make it solve maths problems, build websites, analyse data, send emails, control robots, or train artificial intelligence models. The same fundamental skill — thinking in clear, ordered steps — underlies all of it. That skill is what we will start building this week.

1.2 Why Python?

There are hundreds of programming languages. You may have heard of a few — C, C++, Java, JavaScript, Ruby, Go. Each one was designed with certain goals in mind, and each one is good at certain things. So why have we chosen Python for your first week?

Python was created in 1991 by a Dutch programmer named Guido van Rossum. He wanted a language that was simple to read, easy to learn, and powerful enough to do real work. The

result is a language whose code looks almost like plain English. Compare these two snippets that do exactly the same thing — print the numbers 1 to 5:

```
# Python
for i in range(1, 6):
    print(i)
```

```
// Java (for comparison only – you do not need to learn this)
for (int i = 1; i ≤ 5; i++) {
    System.out.println(i);
}
```

Notice how Python drops the brackets, the semicolons, the type declarations and the curly braces. It uses simple English words (`for` , `in`) and indentation (the four spaces before `print`) to show structure. This makes Python one of the most readable languages ever designed — and readability is exactly what a beginner needs.

Beyond readability, Python is also genuinely powerful. It is the language behind large parts of Google's search engine, Instagram's backend, Netflix's recommendation system, and NASA's mission planning tools. It is the dominant language in data science and machine learning. So when you learn Python, you are not learning a toy language — you are learning a tool that real engineers use to build real products.

— DID YOU KNOW?

Python is not named after the snake. Guido van Rossum was a fan of the British comedy series *Monty Python's Flying Circus*, and chose the name as a short, unique, and slightly mysterious word. The language's official documentation even contains references to spam, eggs, and the knights who say "Ni".

1.3 Where Python is used in the real world

It is natural to wonder, as you start learning, whether the language you are picking up is actually useful outside the classroom. With Python, the answer is an unambiguous yes. Below are some of the major fields where Python plays a central role today.

Field	How Python is used there
Web Development	Frameworks like Django and Flask power backends of Instagram, Spotify, and Dropbox.
Data Science & Analytics	Libraries like pandas, NumPy and matplotlib are the standard tools for analysing data.
Machine Learning & AI	TensorFlow, PyTorch and scikit-learn are all Python-first. Most AI research code is in Python.
Automation & Scripting	System admins write Python scripts to automate file backups, server checks, and reports.
Scientific Computing	Researchers at CERN, ISRO and NASA use Python for simulations and data analysis.
Education	Python is the introductory programming language at MIT, IITs, IISc, and most top universities.

1.4 What you will build this week

Learning by reading alone is slow. Learning by building is fast and lasting. Throughout this week you will build five small but complete programs, each one applying everything you have learned up to that point. These are not toy exercises — each one is a genuine, working application that you could show to a friend.

PROJECT 1

Calculator Program

PROJECT 2

Number Guessing Game

PROJECT 3

Student Management Program

PROJECT 4

Expense Tracker Application

PROJECT 5

Library Management System (Mini Project)

By the end of Project 5, you will have written a small object-oriented application that stores data in a file, handles errors gracefully, and uses classes, functions, loops, conditionals and data structures together. That is a substantial achievement for a single week — and it is entirely within your reach.

1.5 How to use this handbook

This handbook is designed to support you in three different situations. Use it before class to preview what will be taught, so the lecture feels familiar rather than surprising. Use it during class to follow along, take notes in the margins, and try the small examples as the instructor writes them. Use it after class for revision — the Quick Recaps, Remember boxes, and practice questions are placed exactly where you will need them.

Every chapter follows a similar rhythm: a concept is introduced with an analogy, then shown in code, then reinforced with examples, and finally tested with practice questions. Do not skip the practice. Reading code is like reading sheet music — it gives you a sense of the tune, but only writing code makes you a player. Try every example. Modify it. Break it. Fix it. That loop — write, run, break, fix — is how programming enters your bones.

— REMEMBER

- Programming is a skill, not a body of facts. You learn it by doing, not by memorising.
- If your code does not run, that is normal. Every programmer, including the author of this handbook, spends a large part of their day fixing errors.
- Type out every example yourself. Copy-paste teaches your eyes; typing teaches your hands.
- When something does not make sense, slow down and re-read. Re-reading is not a sign of weakness — it is a sign of careful learning.

1.6 The mindset of a beginner programmer

Many students arrive at programming with a hidden worry: *am I smart enough for this?* The answer is yes. Programming is not about being a genius. It is about being patient, being willing to make mistakes, and being willing to keep going when something does not work the first time. Every professional programmer was once a confused beginner staring at an error message they did not understand. The difference between those who became programmers and those who gave up was not talent — it was persistence.

Expect to feel confused sometimes. Expect to make typos. Expect to forget whether it is `print` or `Print` (it matters!). Expect to spend ten minutes looking for a missing colon. These are not signs that you are bad at programming — they are the actual experience of programming. Welcome to the craft.

— TIP — THE 20-MINUTE RULE

When you get stuck on an error, try to fix it yourself for twenty minutes. Read the error message carefully. Search the exact error text online. After twenty minutes, if you are still stuck, ask for help — from a classmate, your instructor, or an AI assistant. Twenty minutes of struggle builds problem-solving muscles; asking immediately builds none.

Practice Questions

CONCEPTUAL CHECK

- Q1.** In one or two sentences of your own, explain what a "programming language" is and why humans cannot simply write instructions in English for the computer to follow directly.
- Q2.** List two real-world products or services (other than the ones mentioned in this chapter) that you suspect might use Python somewhere in their technology stack, and briefly explain why.

— INTERVIEW CORNER

Q1. Why is Python called a "high-level" language?

Hint: Think about how far the language is from the machine's own instructions (0s and 1s).

Q2. Name two advantages of Python over C for a beginner learning to program.

— FUN FACT

The word "Python" appears in the official style guide for Python code, called **PEP 8** — "PEP" stands for Python Enhancement Proposal. PEPs are how the Python community proposes and discusses changes to the language. There are over 8,000 numbered PEPs as of 2025, each one a small piece of Python's history.

QUICK RECAP

- Programming = writing clear step-by-step instructions for a computer.
- Python was created by Guido van Rossum in 1991 and named after Monty Python.
- Python is readable, beginner-friendly, and used in web, data science, AI, automation, and research.
- This week you will build five projects: Calculator, Number Guessing Game, Student Management, Expense Tracker, and a Library Management mini project.
- Use this handbook before, during and after class. Type every example yourself.

CHAPTER 02

Setting Up Your Python Environment

Before you can write a single line of Python, you need a working environment. In this chapter we will install Python itself, set up a code editor called VS Code, and also learn how to use a browser-based tool called Google Colab. By the end, you will be able to write, run and save Python programs on your own machine.

Installation can feel tedious, especially when you are eager to start coding. Be patient. A clean setup done once saves hours of frustration later. We will go step by step, verify each step, and address the common mistakes that catch beginners.

2.1 Three terms you must understand first

Before any installation, you need to know what three words actually mean. Many beginners copy installation commands without understanding them, and then get lost when something goes wrong. Take a minute to understand these.

Term	What it actually means
Python interpreter	A program that reads your Python code line by line and translates it into instructions the computer's processor can execute. Without it, your <code>.py</code> files are just text. When you "install Python", this is what you are installing.
Code editor (IDE)	A program for writing and editing code comfortably — with features like syntax highlighting, auto-indentation, and a "Run" button. VS Code is the most popular free editor today. An IDE (Integrated Development Environment) is a more feature-rich editor.
Runtime / Environment	The place where your code actually runs. This could be your own computer (running the Python interpreter directly) or a remote server (like Google Colab's machines) that runs the code for you and sends back the result.

— ANALOGY

Think of the **Python interpreter** as a translator at a conference. You write a speech in Python (your code). The interpreter reads it aloud in the language the audience (your CPU) understands. The **code editor** is the word processor you used to write the speech. The **runtime** is the conference hall where the speech is delivered. Three different things, three different tools — all needed.

2.2 Installing Python on Windows

These steps assume you are using Windows 10 or 11. The steps for macOS are given in the next section.

1. Open your browser and go to the official Python website:
`https://www.python.org/downloads/`
2. The page will detect your operating system automatically. Click the big yellow button that says **Download Python 3.x.x** (the version number changes over time — any 3.10 or higher is fine for this handbook).
3. A file named `python-3.x.x-amd64.exe` will download. Double-click it to launch the installer.
4. **Critical step:** On the first installer screen, look at the bottom of the window. There is a checkbox that says **"Add python.exe to PATH"**. *Check this box*. If you skip this step, Windows will not know where Python is, and commands like `python` will fail later.
5. Click **"Install Now"**. The installer will copy Python to your system. This takes a minute or two.
6. When the installer says "Setup was successful", click **Close**.

Verifying the installation on Windows

1. Press `Windows + R` on your keyboard to open the Run dialog.
2. Type `cmd` and press Enter. A black Command Prompt window opens.
3. Type the following command and press Enter: `python --version`

4. You should see output like `Python 3.12.4` (your version number may differ slightly). If you see this, Python is installed correctly.
5. Now type: `pip --version` — you should see a line showing the pip version. `pip` is Python's package manager, used to install external libraries later.

OUTPUT

```
C:\Users\student> python --version
Python 3.12.4
```

```
C:\Users\student> pip --version
pip 24.0 from C:\Users\student\AppData\...\pip (python 3.12)
```

— COMMON MISTAKE — "PYTHON IS NOT RECOGNIZED"

If you see `'python' is not recognized as an internal or external command`, it means Python is not in your PATH. The most common cause is skipping the "Add python.exe to PATH" checkbox during installation. **Fix:** Re-run the installer, choose "Modify", and tick the PATH option. Or, manually add Python's install folder to your system's Environment Variables (search "Edit environment variables" in the Windows Start menu).

2.3 Installing Python on macOS

macOS comes with an old version of Python pre-installed, but you should not use that — it is outdated and Apple may remove or change it. Install a fresh copy using one of the two methods below.

Method A — Official installer (easiest for beginners)

1. Go to <https://www.python.org/downloads/> and click **Download Python 3.x.x**.
2. A `.pkg` file will download. Double-click it and follow the installer steps (just keep clicking Continue and Install).
3. Open the **Terminal** app (Cmd + Space, type "terminal", press Enter).
4. Type `python3 --version` and press Enter. On macOS, the command is `python3` (not `python`) to avoid clashing with the old system Python.

Method B — Using Homebrew (for students who already use brew)

```
# Install Homebrew first if you do not have it (https://brew.sh)
brew install python3
python3 --version
```

2.4 Your first Python program — the REPL

Once Python is installed, the simplest way to try it out is the **REPL** — the Read-Eval-Print Loop. The REPL is an interactive window where you type one line of Python, press Enter, and Python runs that line and shows the result immediately. It is perfect for experimenting.

Open your terminal or command prompt and type `python` (or `python3` on macOS). You will see something like this:

OUTPUT

```
C:\Users\student> python
Python 3.12.4 (tags/v3.12.4) on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

The `>>>` is the REPL prompt, waiting for your input. Type your first line of Python:

```
>>> print("Hello, World!")
```

OUTPUT

```
Hello, World!
```

Congratulations — that is your first Python program. The `print()` function tells Python to display whatever is inside the parentheses on the screen. To exit the REPL, type `exit()` or press `Ctrl + Z` then Enter (on Windows) or `Ctrl + D` (on macOS).

— DID YOU KNOW?

The tradition of writing "Hello, World!" as your first program goes back to 1972, when a programmer named Brian Kernighan used it in a tutorial for the B programming language. Since then, almost every programming language tutorial in the world starts with the same two words — a fifty-year-old inside joke that you are now part of.

2.5 Setting up VS Code

The REPL is great for experiments, but real programs are written in files, saved, and run as a whole. For that you need a code editor. We will use **Visual Studio Code** (VS Code) — a free, lightweight editor from Microsoft that has become the most popular code editor in the world.

Step-by-step VS Code setup

1. Go to <https://code.visualstudio.com/> and click the big blue Download button. Choose the version for your operating system (Windows, macOS, or Linux).
2. Run the installer with the default options. On Windows, make sure the option "Add to PATH" is checked.
3. Open VS Code. You will see a welcome screen with a sidebar on the left.
4. Click the **Extensions** icon in the left sidebar (it looks like four squares). Or press `Ctrl + Shift + X` (Windows) / `Cmd + Shift + X` (macOS).
5. In the search box, type "Python". The first result should be the official **Python** extension by Microsoft. Click **Install**.
6. This extension gives VS Code the ability to understand Python syntax, suggest completions, run Python files, and highlight errors as you type.

Writing and running your first Python file in VS Code

1. Create a new folder on your computer (say, `Documents/python-practice`) and open it in VS Code: **File** → **Open Folder**.
2. Click the "New File" icon next to the folder name in the sidebar. Name it `hello.py` . The `.py` extension tells VS Code this is a Python file.
3. Type the following code into the file:

```
# hello.py - my first Python file
name = input("What is your name? ")
print(f"Hello, {name}! Welcome to Python.")
```

4. Save the file with `Ctrl + S` (or `Cmd + S` on macOS).
5. To run the file, click the triangular "Play" button in the top-right corner of the VS Code window. Or open the integrated terminal (`Ctrl + ``) and type `python hello.py` .
6. You will be prompted for your name, and then greeted. You have just run your first saved Python program.

OUTPUT

```
What is your name? Aisha
Hello, Aisha! Welcome to Python.
```

REMEMBER

- Always save Python files with the `.py` extension.
- File names should be lowercase, with no spaces — use underscores if needed (e.g. `my_first_program.py`).
- The "Run" button at the top right of VS Code is the easiest way to execute a Python file. If it is missing, your Python extension is not installed correctly.

2.6 Setting up Google Colab

Google Colab (short for Colaboratory) is a free, browser-based environment for writing and running Python. You do not need to install anything on your computer — Python is already installed on Google's servers, and Colab lets you use it through your browser. Colab is especially popular for data science, machine learning, and for students who share a computer or use a Chromebook.

Step-by-step Colab setup

1. Open your browser and go to `https://colab.research.google.com` . Sign in with any Google account.
2. Click **New Notebook** (or File → New Notebook). A new notebook opens.
3. A Colab notebook is a sequence of **cells**. Each cell can hold either code or text. The first cell is a code cell by default.
4. Type the following into the cell:

```
print("Hello from Colab!")  
print(2 ** 10)
```

5. To run the cell, click the triangular "Play" button on its left, or press **Shift + Enter** .
6. The output appears directly below the cell.

OUTPUT

```
Hello from Colab!  
1024
```

Why two environments?

You might wonder why we set up both VS Code and Colab. They serve different purposes, and a real programmer uses both. The table below compares them so you know when to choose which.

Feature	VS Code	Google Colab
Installation	Needs install on your computer	No install — runs in browser
Internet	Not needed to run code	Required always
File storage	Saved on your computer	Saved in your Google Drive
Best for	Building projects, web apps, scripts, real software	Data science, ML experiments, sharing notebooks, classroom demos
Free GPU?	No	Yes (limited free GPU/TPU access)
Works on Chromebook?	Difficult / not supported	Yes, perfectly
Code runs on	Your computer	Google's servers

For most of this handbook, either environment works fine. We recommend using VS Code on your own computer for practice, because it is what professional Python developers actually use. Use Colab when you cannot install software (for example, in a college computer lab with restrictions), or when you want to share a notebook with a classmate.

2.7 Common installation problems & troubleshooting

Problem	Likely cause & fix
<code>'python'</code> is not recognized on Windows	Python not added to PATH. Re-run installer, choose Modify, tick "Add Python to PATH". Restart terminal.
<code>python</code> opens Microsoft Store on Windows	Windows has a "fake" python alias. Go to Settings → Apps → App execution aliases, and turn off the python.exe and python3.exe aliases.
Multiple Python versions installed	Common on macOS. Always use <code>python3</code> and <code>pip3</code> explicitly. Or use a version manager like pyenv later.
<code>pip</code> command not found	Try <code>pip3</code> instead, or run <code>python -m pip</code> . Re-run the Python installer and ensure pip is selected.
VS Code shows "Select Python Interpreter"	Press <code>Ctrl + Shift + P</code> , type "Python: Select Interpreter", and pick the Python version you installed.
Colab says "Runtime disconnected"	Free Colab sessions disconnect after ~90 minutes of inactivity. Reconnect and re-run your cells. Long-running code should save intermediate results.

Practice Questions

MULTIPLE CHOICE

Q1. What does the Python interpreter actually do?

- A. It edits your code for you automatically.
- B. It reads your Python code line by line and translates it into machine instructions.
- C. It uploads your code to the internet.
- D. It checks your code for spelling mistakes in English.

Q2. Which checkbox must you tick when installing Python on Windows?

- A. "Install Python as administrator"
- B. "Add python.exe to PATH"
- C. "Use Python offline"
- D. "Install Java runtime"

Q3. Which command on macOS runs the installed Python?

- A. `python`
- B. `python3`
- C. `py`
- D. `run python`

SHORT EXERCISE

Q4. Create a new file called `first_program.py` in VS Code. Inside it, write a program that prints your name, your college name, and the current year on three separate lines. Run the file and confirm the output. (No answer key — you verify by running it.)

— INTERVIEW CORNER

Q1. What is the difference between an interpreter and a compiler?

Hint: Think about *when* the translation to machine code happens — all at once, or line by line.

Q2. Why is it recommended to use `python3` instead of `python` on macOS and Linux?

— FUN FACT

Python 2 and Python 3 are not fully compatible — code written in Python 2 may not run in Python 3, and vice versa. Python 3 was released in 2008 to fix design mistakes in Python 2, but it took the community over a decade to fully migrate. Python 2 officially reached end-of-life on January 1, 2020. Always learn Python 3 — which is what you are using.

QUICK RECAP

- Three tools: Python interpreter (runs code), code editor like VS Code (writes code), runtime (where code executes).
- Install Python from python.org; on Windows, always tick "Add python.exe to PATH".
- Verify with `python --version` in terminal.
- VS Code + Python extension is the recommended editor for this handbook.
- Google Colab runs Python in the browser — great for classroom demos and data science.
- If a command fails, the cause is almost always PATH or version mismatch.

HOMework

1. Write a one-paragraph explanation (3–4 sentences) of why a code editor like VS Code is better than a plain text editor (like Notepad) for writing Python.
2. Set up both VS Code and Google Colab. In each, write a program that prints your name and the date. Confirm both work.
3. Explore the VS Code Python extension's features. Find out what keyboard shortcut runs the current Python file in your OS. Write it down for reference.
4. Suppose your friend reports that running `python --version` in CMD gives "command not found". List three things you would check to diagnose the problem.

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (b) It reads your Python code line by line and translates it into machine instructions.
- Q2. (b) "Add python.exe to PATH"
- Q3. (b) `python3`

CHAPTER 03

Variables, Data Types & Input/Output

Now that your environment is ready, it is time to write programs that actually do something useful. In this chapter we learn how to store information in variables, understand what kinds of data Python can work with, take input from the user, and display output neatly. These four ideas — variables, types, input, output — form the foundation of every Python program you will ever write.

3.1 What is a variable?

Imagine you are filling out a form. The form has fields like *Name*, *Age*, *City*. Each field has a label and a value — the label tells you what the value means, and the value is the actual information. A variable in Python works exactly the same way. It is a named container that holds a value, so you can refer to that value later by its name.

Here is the simplest possible example:

```
age = 21
```

This single line does three things. It tells Python: create a container, name it `age`, and store the integer `21` inside it. From this point onwards, whenever you write `age` in your code, Python will look up the value `21` and use it.

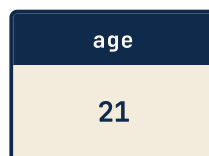


FIGURE 3.1 — A VARIABLE IS A LABELLED BOX HOLDING A VALUE

You can have as many variables as you want, each with its own name and value:


```
name = "Aisha"
age = 21
height_cm = 165.5
is_student = True
```



FIGURE 3.2 — FOUR VARIABLES, EACH HOLDING A DIFFERENT TYPE OF VALUE

3.2 Rules for naming variables

Python gives you a lot of freedom in naming variables, but there are a few strict rules. Break these and your program will not run.

Rule	Valid examples / Invalid examples
Can use letters, digits and underscores	Valid: <code>score</code> , <code>player_1</code> , <code>_temp</code> Invalid: <code>pla\$yer</code> , <code>sc@re</code>
Cannot start with a digit	Valid: <code>user1</code> , <code>a1b2</code> Invalid: <code>1user</code> , <code>2nd_place</code>
Cannot be a Python keyword	Invalid: <code>for</code> , <code>if</code> , <code>class</code> , <code>def</code> , <code>True</code> , <code>return</code>
Case-sensitive	<code>age</code> , <code>Age</code> , <code>AGE</code> are three different variables
No spaces allowed	Invalid: <code>first name</code> Valid: <code>first_name</code>

— REMEMBER — PEP 8 NAMING CONVENTION

Python's official style guide (PEP 8) recommends:

- Use **lowercase with underscores** for variable and function names: `student_name` , `total_marks` .
- Use **UPPER_CASE** for constants: `MAX_RETRIES = 5` , `PI = 3.14159` .
- Use **CapitalisedWords** for class names (we will see this in Chapter 11): `class Student` .
- Names should be **meaningful**. `x = 21` is bad. `age = 21` is good.

3.3 The assignment statement

In Python, the equals sign `=` is called the **assignment operator**. It does not mean "is equal to" the way it does in mathematics. It means: take the value on the right side and store it in the variable on the left side. Read it as an arrow pointing left: `age ← 21` .

```
# Right side is evaluated first, then stored on the left
total = 10 + 20          # total now holds 30
greeting = "Hello, " + "World" # greeting holds "Hello, World"

# You can also re-assign a variable — the old value is gone
score = 50
score = 75      # score is now 75; the 50 is lost

# You can use a variable's current value to update itself
score = score + 10 # score is now 85
```

— COMMON MISTAKE — "NAMEERROR"

Beginners often try to use a variable before assigning it. For example:

```
print(total_marks) # NameError if total_marks was never assigned
```

Why it fails: Python reads your code top-to-bottom. If `total_marks` has not been given a value before this line, Python has nothing to print. **Fix:** always assign a variable before using it. `total_marks = 0` first, then print it.

3.4 Python's basic data types

Every value in Python has a **type** — a classification that tells Python what kind of data it is and what operations can be performed on it. The integer `21` is a different type from the text `"Aisha"`, because you can do arithmetic on the first but not on the second. Below are the five basic types you will use constantly.

Type	What it holds	Example
<code>int</code>	Whole numbers (positive, negative, or zero)	<code>42</code> , <code>-7</code> , <code>0</code> , <code>1000000</code>
<code>float</code>	Numbers with a decimal point	<code>3.14</code> , <code>-0.001</code> , <code>2.0</code>
<code>str</code>	Text — a sequence of characters in quotes	<code>"Hello"</code> , <code>'Python'</code> , <code>"123"</code>
<code>bool</code>	Truth values — only <code>True</code> or <code>False</code>	<code>True</code> , <code>False</code>
<code>NoneType</code>	"Nothing" — absence of a value	<code>None</code>

Note that `"123"` (in quotes) is a string, not an integer — even though it looks like a number. Python does not perform arithmetic on it the same way it would on the integer `123`. We will see this distinction cause real bugs shortly.

3.5 The `type()` function and dynamic typing

Python is a **dynamically typed** language. This means you do not have to declare the type of a variable before using it — Python figures out the type automatically based on the value you assign. The same variable can even hold different types at different times (though this is usually bad practice).

```
x = 10          # x is an int
print(type(x))  # <class 'int'>

x = "hello"     # now x is a str
print(type(x))  # <class 'str'>

x = 3.14        # now x is a float
print(type(x))  # <class 'float'>
```

OUTPUT

```
<class 'int'>
<class 'str'>
<class 'float'>
```

The `type()` function is your friend — whenever you are unsure what type a value is, print its type and Python will tell you. This is one of the most useful debugging tools for beginners.

3.6 Type conversion (casting)

Sometimes you have a value of one type and need it in another. For example, the user might type their age as text, but you need it as a number to do arithmetic. Python provides built-in functions to convert between types:

Function	Converts to	Example
<code>int()</code>	integer	<code>int("42")</code> → 42 <code>int(3.9)</code> → 3 (truncates, does not round)
<code>float()</code>	float	<code>float("3.14")</code> → 3.14 <code>float(5)</code> → 5.0
<code>str()</code>	string	<code>str(42)</code> → "42" <code>str(3.14)</code> → "3.14"
<code>bool()</code>	boolean	<code>bool(0)</code> → False <code>bool("anything")</code> → True

```
# Explicit conversion
age_text = "21"
age_number = int(age_text)
print(age_number + 5)      # 26 - arithmetic works

# Implicit conversion (Python does this for you)
result = 10 + 2.5
print(result, type(result))  # 12.5 <class 'float'>
```

— COMMON MISTAKE — STRING + NUMBER

```
age = "21"
print(age + 5)    # TypeError!
```

Why: `age` is a string here, and you cannot add a string to an integer. Python does not auto-convert in this case. **Fix:** convert explicitly: `print(int(age) + 5)` → `26`.

3.7 Output with `print()`

The `print()` function is Python's main way of showing output to the user. It can take one value, several values, or even no value at all.

```
# Print a single value
print("Hello, World!")

# Print multiple values - separated by spaces by default
print("Name:", "Aisha", "Age:", 21)

# Use sep= to change the separator
print("2025", "08", "15", sep="-")

# Use end= to change what is printed at the end (default is newline)
print("Loading", end=" ")
print("done")
```

OUTPUT

```
Hello, World!
Name: Aisha Age: 21
2025-08-15
Loading done
```

f-strings — the modern way

Since Python 3.6, the cleanest way to combine text and values is the **f-string**. Put an `f` before the opening quote, and any expression inside curly braces `{}` gets evaluated and inserted into the string.

```
name = "Aisha"
age = 21
height = 165.5

print(f"My name is {name}, I am {age} years old.")
print(f"Next year I will be {age + 1}.")
print(f"My height is {height:.1f} cm.") # 1 decimal place
```

OUTPUT

```
My name is Aisha, I am 21 years old.
Next year I will be 22.
My height is 165.5 cm.
```

REMEMBER

- f-strings start with the letter `f` immediately before the opening quote: `f" ... "`
- Anything inside `{}` is treated as Python code, evaluated, and inserted into the string.
- Use `{value:.2f}` to format a float to 2 decimal places.
- f-strings are the recommended way to format output in modern Python.

3.8 Input with `input()`

The `input()` function lets your program ask the user for information. Whatever the user types is returned to your program **as a string** — even if they typed a number. This is the single biggest source of beginner bugs.

```
name = input("Enter your name: ")
print(f"Hello, {name}!")

age = input("Enter your age: ")
print(f"You typed: {age}")
print(type(age)) # <class 'str'> – even though they typed a number
```

OUTPUT

```
Enter your name: Aisha
Hello, Aisha!
Enter your age: 21
You typed: 21
<class 'str'>
```

To get a number from the user, wrap `input()` in a conversion function:

```
age = int(input("Enter your age: "))
print(f"Next year you will be {age + 1}.")

price = float(input("Enter the price: "))
print(f"With 18% GST: Rs. {price * 1.18:.2f}")
```

OUTPUT

```
Enter your age: 21
Next year you will be 22.
Enter the price: 500
With 18% GST: Rs. 590.00
```

— COMMON MISTAKE — FORGETTING TO CONVERT

```
num1 = input("Enter first number: ")    # "10"
num2 = input("Enter second number: ")   # "20"
print(num1 + num2)                      # "1020" - string concatenation, not addition!
```

Why: `input()` always returns a string, so `+` joins them like text instead of adding like numbers. **Fix:** `num1 = int(input(...))` for both, then `print(num1 + num2)` gives `30`.

3.9 Comments

A **comment** is a note in your code that Python ignores. Comments exist for humans — to explain what the code does, why a particular approach was taken, or to mark a section that needs work. Use them generously; your future self will thank you.

```
# This is a single-line comment - Python ignores everything after #

# Calculate the area of a rectangle
length = float(input("Length: "))
width = float(input("Width: "))
area = length * width    # area = length times width
print(f"Area = {area}")

"""
This is a multi-line comment (technically a docstring).
Useful for longer explanations.
Python ignores everything between the triple quotes.
"""
```

Practice Questions

MULTIPLE CHOICE

Q1. Which of the following is a valid variable name in Python?

- A. `2nd_place`
- B. `student-name`
- C. `_score`
- D. `class`

Q2. What is the type of the value `"42"` ?

- A. `int`
- B. `str`
- C. `float`
- D. `bool`

OUTPUT PREDICTION

Q3. What does the following code print?

```
x = 7
y = 3
print(f"{x} divided by {y} is {x / y:.2f}")
```

Q4. What is the output of this code?

```
a = "5"
b = "3"
print(a + b)
print(int(a) + int(b))
```

FILL IN THE BLANKS

Q5. The `_____` function is used to find out what type a value is.

Q6. The `input()` function always returns a value of type `_____`.

DEBUGGING

Q7. Find and fix the error in this code:

```
age = input("Enter your age: ")
print(f"In 5 years you will be {age + 5}")
```

SHORT CODING EXERCISES

Q8. Write a program that asks the user for their name and their favourite programming language, then prints: `Hi <name>! You love <language>.`

Q9. Write a program that asks the user for the radius of a circle, then prints the area (`pi * r * r`) rounded to 2 decimal places. Use `3.14159` for pi.

— INTERVIEW CORNER

Q1. What is the difference between `=` and `==` in Python?

Q2. Why does Python say `21` and `"21"` are different? What type is each?

Q3. What does it mean when we say Python is "dynamically typed"? Give one advantage and one risk.

— FUN FACT

Python's integers have no maximum size. In languages like C or Java, an `int` cannot hold a number larger than about 2 billion. In Python, you can write `print(2 ** 1000)` and Python will happily print a 302-digit number. This is because Python integers grow in memory as needed — a small but genuine superpower.

QUICK RECAP

- A variable is a labelled container that holds a value. Use `name = value` to assign.
- Variable names: letters, digits, underscores; cannot start with a digit; case-sensitive; no keywords.
- Five basic types: `int`, `float`, `str`, `bool`, `NoneType`.
- Use `type()` to check a value's type. Python is dynamically typed.
- Convert with `int()`, `float()`, `str()`, `bool()`.
- `input()` always returns a string. Convert before doing arithmetic.
- f-strings (`f" ...{expr} ..."`) are the modern way to format output.
- Comments start with `#`. Use them to explain your code to humans.

HOMework

1. Write a program that asks the user for two numbers, then prints their sum, difference, product, and quotient (all on separate lines with clear labels).
2. Write a program that asks the user for their name, age, and height in cm. Then print a single sentence: *"My name is X, I am Y years old and Z cm tall."* using an f-string.
3. Write a program that asks the user for the price of an item and the quantity purchased. Calculate the total price, apply 18% GST, and print the final amount rounded to 2 decimals.
4. Write a program that swaps the values of two variables without using a third variable. (Hint: Python supports tuple unpacking — `a, b = b, a`.)
5. A student wrote the following code and got an error. Explain in 2–3 sentences what went wrong and how to fix it: `age = input("Age: "); print(age + 5)`

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (c) `_score`
- Q2. (b) `str`
- Q3. `7 divided by 3 is 2.33`
- Q4. `53` (string concatenation) then `8` (integer addition)
- Q5. `type()`
- Q6. `str` (string)
- Q7. `input()` returns a string, so `age + 5` fails. Fix: `age = int(input("Enter your age: "))`

CHAPTER 04

Operators in Python

Operators are the verbs of programming. If variables and values are the nouns, operators are the actions that combine them, compare them, and produce new values. You already know many operators from school mathematics — `+`, `-`, `*`, `/` — and Python uses those same symbols. Python also adds a few powerful new ones, which we will meet in this chapter.

4.1 Arithmetic operators

Arithmetic operators perform mathematical operations on numbers. Below is the full set you will use, along with examples and the result each produces.

Op	Name	Example	Result
+	Addition	7 + 3	10
-	Subtraction	7 - 3	4
*	Multiplication	7 * 3	21
/	True division	7 / 3	2.3333333 (always float)
//	Floor division	7 // 3	2 (whole part only)
%	Modulus (remainder)	7 % 3	1 (remainder)
**	Exponentiation	2 ** 3	8 (2 to the power 3)

The three that surprise beginners are `/`, `//` and `%`. Let us look at each carefully.

True division `/` vs floor division `//`

The single slash `/` always returns a **float**, even if the result is a whole number. The double slash `//` returns the integer part of the division, throwing away any fractional part. This is sometimes called "integer division".

```
print(10 / 2)    # 5.0    - float, even though result is whole
print(10 // 2)   # 5      - integer
print(7 / 2)     # 3.5
print(7 // 2)    # 3      - decimal part discarded
print(-7 // 2)   # -4     - rounds towards negative infinity, not zero
```

The modulus operator `%`

The modulus operator returns the **remainder** after division. It is surprisingly useful. The two most common uses are: (a) checking if a number is even or odd, and (b) finding the last digit of a number.

```
# Checking even / odd
print(14 % 2)    # 0 - so 14 is even
print(17 % 2)    # 1 - so 17 is odd

# Last digit of a number
print(4567 % 10) # 7 - the last digit

# Checking divisibility
print(100 % 25)  # 0 - so 100 is divisible by 25
```

4.2 Comparison operators

Comparison operators compare two values and produce a **boolean** result — either `True` or `False`. They are the foundation of decision-making in Python, which we will explore fully in the next chapter.

Op	Meaning	Example	Result
<code>=</code>	Equal to	<code>5 = 5</code>	True
<code>≠</code>	Not equal to	<code>5 ≠ 3</code>	True
<code><</code>	Less than	<code>3 < 5</code>	True
<code>></code>	Greater than	<code>7 > 9</code>	False
<code>≤</code>	Less than or equal to	<code>5 ≤ 5</code>	True
<code>≥</code>	Greater than or equal to	<code>6 ≥ 8</code>	False

— COMMON MISTAKE — `=` VS `==`

This is the most common bug in beginner Python. A single `=` **assigns** a value. A double `==` **compares** two values. Mixing them up causes either an error or a silently wrong program.

```
# WRONG - this assigns 5 to age, does not compare
if age = 18:           # SyntaxError
    print("adult")

# CORRECT - compare with ==
if age == 18:
    print("adult")
```

4.3 Logical operators

Logical operators combine multiple boolean expressions into a single result. There are three: `and`, `or`, `not`. Python uses English words for these (not symbols like `&&` or `||` as some other languages do).

Op	Meaning	Example
<code>and</code>	True only if <i>both</i> sides are True	<code>(age ≥ 18) and (has_ticket)</code>
<code>or</code>	True if <i>at least one</i> side is True	<code>(is_member) or (is_vip)</code>
<code>not</code>	Reverses True/False	<code>not is_raining</code>

Truth table

The clearest way to understand `and` / `or` is a truth table — a table showing every possible combination of inputs and the resulting output.

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

— REAL-LIFE ANALOGY

Imagine the entry rule for a movie: "You can enter if you are 18 or older **AND** you have a ticket." Both conditions must be true. If either is false, you cannot enter. That is **and**.

Now imagine a buffet offer: "You get a discount if you are a student **OR** a senior citizen." Either condition being true is enough. That is **or**.

And **not** simply flips the answer: **not True** is **False**, **not False** is **True**.

4.4 Assignment operators

You already know the basic assignment operator `=`. Python also provides **compound assignment operators** — shortcuts that combine an operation with an assignment. They save typing and are very common in real code.

Operator	Equivalent to	Example
<code>=</code>	Simple assignment	<code>x = 5</code>
<code>+=</code>	<code>x = x + n</code>	<code>x += 3</code> → x becomes 8
<code>-=</code>	<code>x = x - n</code>	<code>x -= 2</code> → x becomes 3
<code>*=</code>	<code>x = x * n</code>	<code>x *= 4</code> → x becomes 20
<code>/=</code>	<code>x = x / n</code>	<code>x /= 5</code> → x becomes 1.0
<code>//=</code>	<code>x = x // n</code>	<code>x //= 2</code> → floor division
<code>%=</code>	<code>x = x % n</code>	<code>x %= 3</code> → remainder
<code>**=</code>	<code>x = x ** n</code>	<code>x **= 2</code> → square

```
# Counting up
total = 0
total += 10    # total is now 10
total += 20    # total is now 30
total += 5     # total is now 35
print(total)   # 35

# Doubling repeatedly
population = 100
population *= 2    # 200
population *= 2    # 400
```

4.5 Operator precedence

When an expression has multiple operators, which one does Python evaluate first? Just like in school mathematics, Python follows a precedence order. The mnemonic **BODMAS** / **PEMDAS** from school mostly applies, with a couple of Python-specific additions.

Order	Operator	Notes
1 (highest)	<code>()</code> — parentheses	Always evaluated first. Use them to override precedence.
2	<code>**</code> — exponentiation	Right-to-left associativity
3	<code>-</code> <code>+</code> (unary, e.g. <code>-x</code>)	Negation
4	<code>*</code> <code>/</code> <code>//</code> <code>%</code>	Multiplication and division, left-to-right
5	<code>+</code> <code>-</code> (binary)	Addition and subtraction, left-to-right
6	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>=</code> <code>!=</code>	Comparisons
7	<code>not</code>	Logical NOT
8	<code>and</code>	Logical AND
9 (lowest)	<code>or</code>	Logical OR

```
print(2 + 3 * 4)      # 14 (3*4 first, then +2)
print((2 + 3) * 4)    # 20 (parentheses force 2+3 first)
print(2 ** 3 ** 2)    # 512 (right-to-left: 3**2=9, then 2**9)
print(10 - 5 - 2)     # 3 (left-to-right: (10-5)-2)
print(5 > 3 and 2 < 8) # True (both comparisons first, then and)
```

— REMEMBER

When in doubt about precedence, **use parentheses**. They make the order explicit and make your code easier to read. Writing `(a + b) * c` is always clearer than relying on the reader to remember precedence rules. Parentheses never hurt performance.

Practice Questions

MULTIPLE CHOICE

Q1. What is the result of `17 % 5` ?

- A. `3`
- B. `2`
- C. `3.4`
- D. `4`

Q2. What is the result of `7 // 2` ?

- A. `3.5`
- B. `3`
- C. `4`
- D. `2`

OUTPUT PREDICTION

Q3. What does this print?

```
print(2 ** 3 ** 2)
```

Q4. What does this print?

```
print(10 > 5 and 3 > 7)
```

Q5. What does this print?

```
x = 5
x *= 3
x += 2
print(x)
```

TRUE / FALSE

- Q6. `/` always returns a `float`, even when both operands are integers and the result is a whole number.
- Q7. `=` and `==` can be used interchangeably in Python.

DEBUGGING

- Q8. This code was meant to check if a number is even, but it always prints "Even" for any number. Find the bug:

```
num = 7
if num % 2 = 0:
    print("Even")
else:
    print("Odd")
```

SHORT CODING EXERCISES

- Q9. Write a one-line expression to compute the average of three numbers `a`, `b`, `c`.
- Q10. Given a two-digit integer `n` (e.g. `47`), write expressions to extract its tens digit and units digit separately.

— INTERVIEW CORNER

- Q1. Explain the difference between `/`, `//`, and `%` with one example each.
- Q2. What is short-circuit evaluation in `and` / `or`? Why does Python use it?

— FUN FACT

The modulus operator `%` has a surprising role in cryptography. The RSA encryption algorithm — used to secure most internet traffic — relies heavily on modular arithmetic. Every time you visit a website with HTTPS, your browser is computing expressions like `(m ** e) % n` with numbers hundreds of digits long. Your Week 1 knowledge of `%` is, in a small way, the foundation of internet security.

QUICK RECAP

- Arithmetic: `+` `-` `*` `/` `//` `%` `**` . Note: `/` always returns float, `//` returns integer division, `%` returns remainder.
- Comparison: `=` `≠` `<` `>` `≤` `≥` — all return `True` or `False` .
- Logical: `and` , `or` , `not` — combine boolean expressions.
- Compound assignment: `+=` `-=` `*=` `/=` `%=` `**=` — shortcuts.
- Precedence: parentheses `>` `**` `>` unary `-+` `>` `*` `/` `//` `%` `>` `+` `-` `>` comparisons `>` `not` `>` `and` `>` `or` .
- When unsure, use parentheses — they are free and remove ambiguity.
- Never confuse `=` (assign) with `==` (compare).

HOMework

1. Write a program that takes a number from the user and prints whether it is even or odd. (Use `%` .)
2. Write a program that takes a 3-digit number and prints the sum of its digits. (Hint: use `//` and `%` with 10 and 100.)
3. Write a program that converts a temperature in Celsius to Fahrenheit, using the formula `F = (C * 9/5) + 32` . Round the result to 1 decimal place.
4. Write a program that asks the user for their marks in 5 subjects, calculates the percentage, and prints "Pass" if percentage is 40 or above, otherwise "Fail". (You may use a single comparison — no `if` needed; we will revisit this with proper control flow.)
5. Without writing any code, evaluate by hand: `2 + 3 * 4 - 6 / 2` . Then verify in Python. Did precedence work as you expected?

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (b) `2` (17 divided by 5 = 3 remainder 2)
- Q2. (b) `3` (floor division discards the decimal)
- Q3. `512` — `3**2 = 9` first (right-to-left), then `2**9 = 512`
- Q4. `False` — `10 > 5` is True, but `3 > 7` is False; True and False = False
- Q5. `17` — `x=5`, then `x=15`, then `x=17`
- Q6. True
- Q7. False — `=` assigns, `==` compares
- Q8. Should be `==` not `=` : `if num % 2 == 0:`

Calculator Program

Chapter Project 1

A Simple Calculator in Python

Your first complete Python program — bringing together variables, input, output, type conversion and operators.

You have learned the building blocks: variables, types, input, output and operators. It is time to assemble them into a real, working program. In this project you will build a calculator that takes two numbers and an operator from the user, then performs the calculation and displays the result.

Project introduction

A calculator is the perfect first project because it uses every concept from the previous chapters in a single program. You will take input (the two numbers and the operation), store values in variables, convert types, perform arithmetic with operators, and display formatted output — everything we have learned so far.

There is one small twist. We have not yet learned control flow (if/elif/else), so we cannot yet ask the user "which operation?" and choose what to do based on their answer. That comes in the next chapter. For now, we will build a calculator that performs *all four basic operations* on the two numbers and shows all four results. After we learn control flow, we will revisit this project and let the user pick a single operation.

Objective

Build a Python program that:

1. Takes two numbers from the user.
2. Performs addition, subtraction, multiplication and division on them.
3. Displays all four results neatly, with proper formatting.
4. Handles the division-by-zero case with a friendly message.

Concepts used

CONCEPTS

Variables

CONCEPTS

input() & print()

CONCEPTS

Type conversion (float)

CONCEPTS

Arithmetic operators

CONCEPTS

f-strings

CONCEPTS

Modulus (for zero check)

Algorithm — the logical approach

STEP-BY-STEP PLAN

- 1 Ask the user for the first number. Convert the input (which is a string) to a `float` and store in `num1`.
- 2 Ask the user for the second number. Convert and store in `num2`.
- 3 Compute `addition = num1 + num2`, `subtraction = num1 - num2`, `multiplication = num1 * num2`.
- 4 For division, first check if `num2` is zero (we will use a simple technique that avoids `if` — for now, we will compute the division safely using a small trick we explain below).
- 5 Print all four results in a clean, labelled format using f-strings.

For the division-by-zero issue, here is the trick. Since we have not yet learned `if`, we cannot check whether `num2` is zero before dividing. We have two options: (a) attempt the division and let Python crash with a `ZeroDivisionError`, or (b) use the fact that `num2 == 0` is itself a boolean expression — we can compute the result conditionally using a Python feature called a **conditional expression**, which is a one-line "if-else".

— TIP — CONDITIONAL EXPRESSIONS (PREVIEW)

Python has a one-line form: `value_if_true if condition else value_if_false`. We will use this here to safely handle division without a full `if` block. It is a small preview of what is coming in the next chapter.

The complete Python code

```
# calculator.py – a simple four-operation calculator

# 1. Take two numbers from the user
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

# 2. Perform the four operations
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2

# 3. Division with zero-check (using a conditional expression)
division = (num1 / num2) if num2 != 0 else "undefined (cannot divide by zero)"

# 4. Display the results
print("\n--- Results ---")
print(f"{num1} + {num2} = {addition}")
print(f"{num1} - {num2} = {subtraction}")
print(f"{num1} * {num2} = {multiplication}")
print(f"{num1} / {num2} = {division}")
```

Code explanation — line by line

Let us walk through the code carefully. Even though it is short, every line teaches something.

Lines 4–5 — taking input: We use `float(input( ... ))` to get a number from the user. The `input()` function reads what the user types as a string; `float()` converts that string into a decimal number. We choose `float` instead of `int` because the user may want to enter decimals like `2.5`.

Lines 8–10 — arithmetic: Three straightforward computations. Each one uses the operator you learned about in Chapter 4. The results are stored in clearly-named variables so the next part of the code is easy to read.

Line 13 — safe division: This is the clever bit. The expression `num2 != 0` evaluates to `True` or `False`. If it is `True` (i.e. `num2` is not zero), Python computes `num1 / num2` normally. If it is `False` (`num2` is zero), Python uses the string `"undefined ..."` instead — avoiding the crash. This is the **conditional expression** we previewed above.

Lines 16–20 — output: We use f-strings to format the output. Each line shows the full calculation (`num1 + num2 = result`) so the user can see exactly what was computed. The `\n` in line 16 inserts a blank line before the results section, making the output easier to read.

Sample input & output

SAMPLE INPUT 1

```
> Enter the first number: 12
> Enter the second number: 4
```

SAMPLE OUTPUT 1

```
--- Results ---
12.0 + 4.0 = 16.0
12.0 - 4.0 = 8.0
12.0 * 4.0 = 48.0
12.0 / 4.0 = 3.0
```

SAMPLE INPUT 2

```
> Enter the first number: 10
> Enter the second number: 0
```

SAMPLE OUTPUT 2

```
--- Results ---
10.0 + 0.0 = 10.0
10.0 - 0.0 = 10.0
10.0 * 0.0 = 0.0
10.0 / 0.0 = undefined (cannot divide by
zero)
```

Possible improvements

Once you have the basic calculator working, here are some ways you could extend it. Some of these will become easy after the next chapter on control flow.

- **Let the user choose the operation:** Ask the user which operation they want (+, -, *, /) and perform only that one. This requires `if/elif/else` — coming up in Chapter 6.
- **Loop for repeated calculations:** After showing the result, ask "Calculate another? (y/n)" and loop back. This requires a `while` loop — coming up in Chapter 7.
- **Power and modulus:** Add `**` (power) and `%` (modulus) as additional operations.
- **Round results:** Currently the output shows `16.0`. You could format floats to 2 decimal places using `{addition:.2f}` in the f-string.
- **Handle invalid input:** What if the user types "hello" instead of a number? Right now Python crashes. Handling this gracefully requires `try/except` — coming up in Chapter 10.

— PRACTICE CHALLENGE

Modify the calculator so that it also prints two extra lines: one showing `num1 ** num2` (num1 raised to the power of num2), and one showing `num1 % num2` (the remainder when num1 is divided by num2). Test your modified program with `num1 = 7` and `num2 = 3`.

— PROJECT REFLECTION

Notice how a real program, even a simple one, requires combining many small ideas: input, conversion, arithmetic, conditional safety, and formatted output. This is what programming *is* — combining small, well-understood pieces into a working whole. As you learn more pieces (control flow, loops, functions, data structures, classes), your programs will become more powerful, but the basic activity stays the same.

Control Flow — Making Decisions

Every program you have written so far has run from top to bottom, line by line, with no choices. Real programs are not that simple. They need to make decisions: *if the user is over 18, allow access; otherwise show an error. If the password is correct, log them in; otherwise try again.* This chapter teaches you how to give your programs the ability to choose between different paths.

6.1 Why programs need to make decisions

Think about your daily routine. When you wake up, you do not follow a fixed sequence of actions regardless of conditions. You check the time, look at the weather, and decide what to do. *If it is raining, take an umbrella. If it is before 7 am, go back to sleep. Otherwise, start your day.* Decisions are everywhere in real life, and they are everywhere in programs too.

In Python, decisions are made using **conditional statements** — the `if`, `elif`, and `else` keywords. They allow your program to evaluate a condition (something that is True or False) and run different code depending on the answer.

6.2 The `if` statement

The simplest decision is a single `if`. It says: *"if this condition is True, run the following block of code. If it is False, skip the block."*

Syntax

```
if condition:
    # code to run if condition is True
    statement_1
    statement_2
# code here runs regardless (not inside the if block)
```

Notice two things: the colon `:` at the end of the `if` line, and the indentation (four spaces) of the statements inside the block. Both are mandatory in Python. The colon tells Python "here comes a block", and the indentation tells Python which lines belong to the block.

Example — age check

```
age = int(input("Enter your age: "))

if age ≥ 18:
    print("You are eligible to vote.")
    print("Please carry your voter ID.")

print("Thank you.")    # always runs
```

OUTPUT

```
Enter your age: 21
You are eligible to vote.
Please carry your voter ID.
Thank you.
```

OUTPUT

```
Enter your age: 15
Thank you.
```

In the second run, the user is 15, so the condition `age ≥ 18` is False. Python skips the two indented lines and jumps directly to the final `print("Thank you.")`.

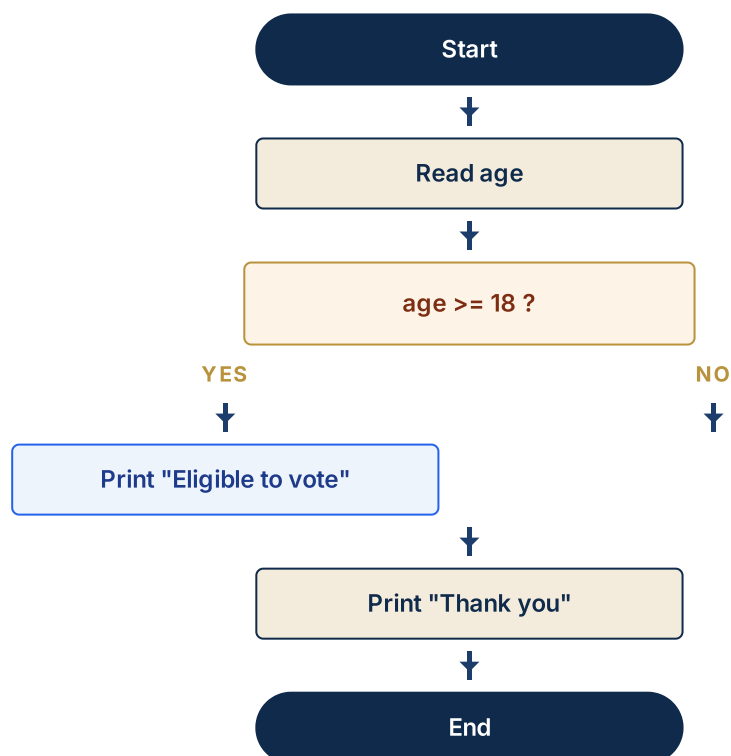


FIGURE 6.1 — FLOWCHART OF A SIMPLE IF STATEMENT

6.3 The if-else statement

A plain `if` runs code only when the condition is True, and does nothing otherwise. Often, you want a different action when the condition is False. That is what `else` is for.

```
number = int(input("Enter a number: "))

if number % 2 == 0:
    print(f"{number} is even.")
else:
    print(f"{number} is odd.")
```

OUTPUT

Enter a number: 7
7 is odd.

OUTPUT

Enter a number: 14
14 is even.

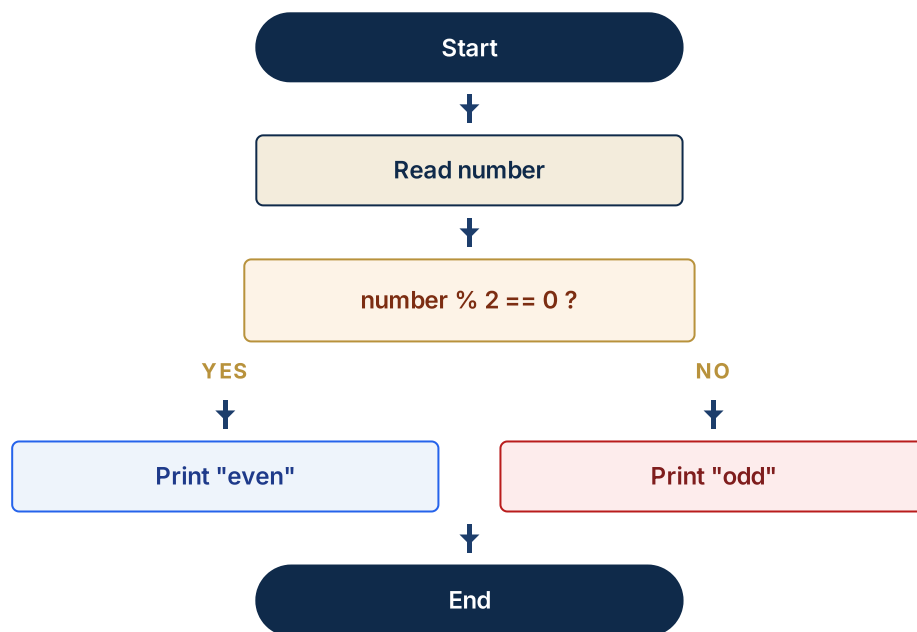


FIGURE 6.2 — FLOWCHART OF AN IF-ELSE STATEMENT

6.4 The if-elif-else statement

When you have more than two possible paths, you use `elif` (short for "else if"). Python checks conditions top to bottom and runs the first one that is True. If none is True, the `else` block runs (if present).

Example — grade calculator

```
marks = int(input("Enter your marks (0-100): "))

if marks ≥ 90:
    grade = "A"
elif marks ≥ 80:
    grade = "B"
elif marks ≥ 70:
    grade = "C"
elif marks ≥ 60:
    grade = "D"
else:
    grade = "F"

print(f"Your grade is: {grade}")
```

OUTPUT

```
Enter your marks (0-100): 85
Your grade is: B
```

Note that we do not need to write `marks < 90` and `marks ≥ 80` for the second condition. Because Python checks top-to-bottom, by the time it reaches the `elif marks ≥ 80` line, we already know `marks < 90` (otherwise the first `if` would have been True and we would not be here).

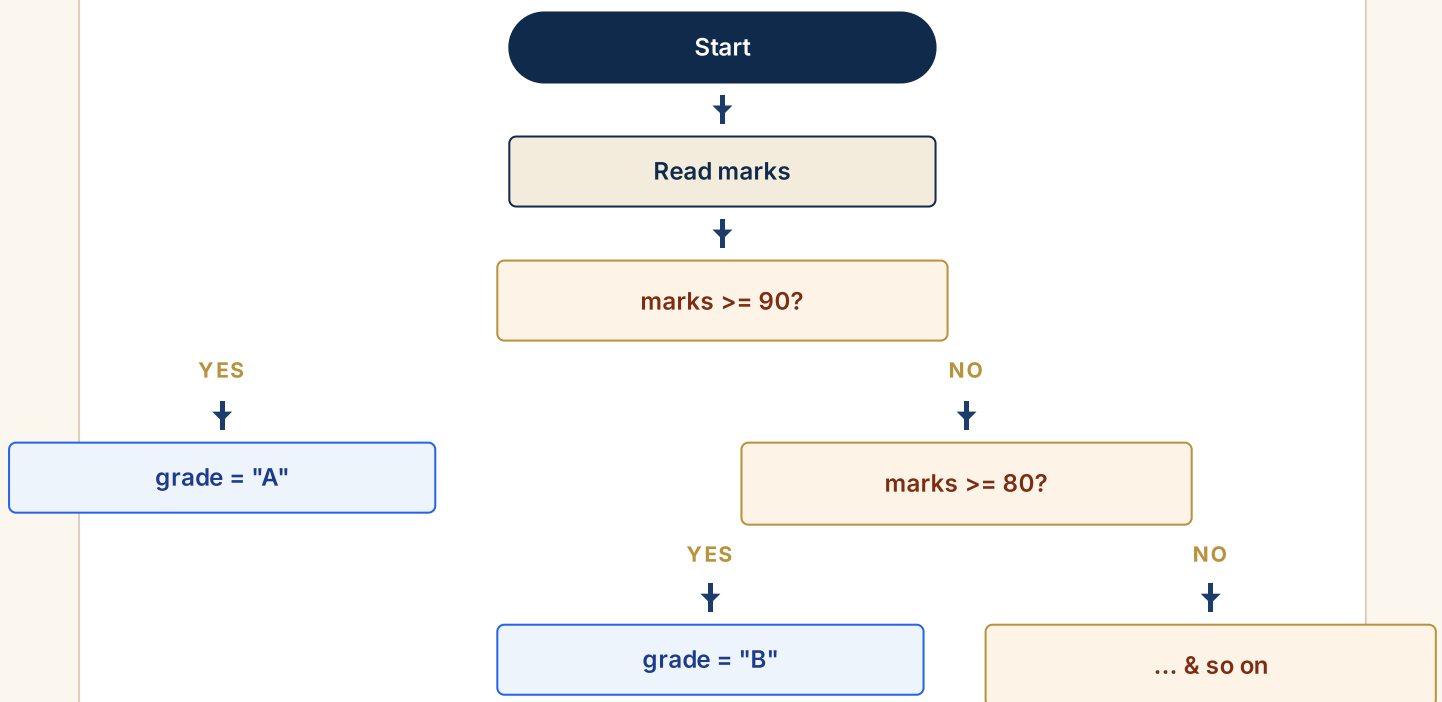


FIGURE 6.3 — FLOWCHART OF AN IF-ELIF-ELSE CHAIN

6.5 Nested `if` statements

You can place an `if` inside another `if`. This is called **nesting**, and it is useful when a decision depends on multiple conditions in sequence.

```
age = int(input("Age: "))
has_ticket = input("Do you have a ticket? (yes/no): ").lower() == "yes"

if age >= 18:
    if has_ticket:
        print("Welcome to the concert!")
    else:
        print("Please buy a ticket at the counter.")
else:
    print("Minors are not allowed.")
```

Notice the doubled indentation for the innermost `print` statements — eight spaces, because they are inside two levels of `if`. Deeply nested code can become hard to read; in practice, you should try to flatten it using `and` / `or` where possible.

6.6 Indentation — Python's defining feature

In most programming languages, blocks of code are marked with curly braces `{ }`. Python is unusual — it uses **indentation** to mark blocks. Four spaces of indentation mean "this line belongs to the block of the statement above". This forces every Python programmer to write visually clean code, which is one of the reasons Python is so readable.

```
# Correct indentation
if x > 0:
    print("positive")      # 4 spaces - inside if
    print("good")         # 4 spaces - inside if
print("done")             # 0 spaces - outside if, always runs
```

— COMMON MISTAKE — INCONSISTENT INDENTATION

```
if x > 0:
    print("positive")
    print("good")  # IndentationError - 6 spaces, not 4
```

Why it fails: Python expects consistent indentation within a block. Mixing 4 and 6 spaces causes an `IndentationError`. **Fix:** use exactly 4 spaces (or one tab) consistently. VS Code auto-indents for you — let it.

— REMEMBER — TWO THINGS EVERY IF NEEDS

- A colon `:` at the end of the `if` line.
- Consistent 4-space indentation for the block underneath.
- Forget either, and Python will complain with a `SyntaxError` or `IndentationError`.

Practice Questions

MULTIPLE CHOICE

Q1. Which keyword is used to check multiple conditions in a chain?

- A. `elseif`
- B. `elif`
- C. `else if`
- D. `elifin`

Q2. What happens if you forget the colon at the end of an `if` line?

- A. Python runs the code anyway.
- B. Python raises a `SyntaxError`.
- C. Python treats the next line as not part of the `if`.
- D. Python inserts the colon automatically.

OUTPUT PREDICTION

Q3. What is printed?

```
x = 10
if x > 5:
    print("A")
if x > 3:
    print("B")
else:
    print("C")
```

Q4. What is printed?

```
n = 7
if n > 10:
    print("big")
elif n > 5:
    print("medium")
elif n > 2:
    print("small")
else:
    print("tiny")
```

DEBUGGING

Q5. This code has an indentation error. Fix it:

```
if 5 > 3:  
print("yes")
```

Q6. This code always prints "Pass" regardless of marks. Find the bug:

```
marks = 35  
if marks = 40:  
    print("Pass")  
else:  
    print("Fail")
```

SHORT CODING

Q7. Write a program that asks the user for a number and prints "Positive", "Negative", or "Zero".

Q8. Write a program that asks for a year and prints whether it is a leap year. (A year is a leap year if it is divisible by 4, except for end-of-century years which must be divisible by 400. So 2000 was a leap year, but 1900 was not.)

— INTERVIEW CORNER

Q1. What is the difference between `if` / `elif` / `else` and a series of independent `if` statements? When would you use which?

Q2. Why does Python use indentation instead of braces? What are the advantages and disadvantages?

Q3. Can an `if` statement have multiple `elif` blocks? Can it have multiple `else` blocks?

— FUN FACT

Python is one of very few mainstream languages that uses indentation for code blocks. Most others (C, Java, JavaScript, Go) use curly braces `{ }`. This design choice was controversial when Python was first released in 1991, but it has won many fans — it forces every Python program to be visually readable, and eliminates an entire class of "missing brace" bugs. Some languages since, like Nim and CoffeeScript, have followed Python's lead.

QUICK RECAP

- `if condition:` — run block if condition is True.
- `if-else` — two paths: one for True, one for False.
- `if-elif-else` — multiple paths; Python checks top-down and runs the first True one.
- Every `if` / `elif` / `else` line ends with a colon `:`.
- Blocks are marked by 4-space indentation (no braces).
- `if` statements can be nested for multi-level decisions.
- Conditions are built using comparison and logical operators.

HOMework

1. Write a program that asks the user for three numbers and prints the largest of the three. (No `max()` function — use `if` / `elif` / `else`.)
2. Write a simple "login" program: ask the user for a username and password. If the username is "admin" and the password is "python123", print "Login successful". Otherwise print "Invalid credentials".
3. Write a program that asks the user for a number from 1 to 7 and prints the corresponding day of the week (1=Monday, 7=Sunday).
4. Write a program that takes a person's height in cm and weight in kg, computes their BMI (`weight / (height/100)**2`), and prints their category: Underweight (<18.5), Normal (18.5–24.9), Overweight (25–29.9), Obese (≥30).
5. Re-implement the Calculator from Project 1, but this time let the user choose the operation (+, -, *, /) using `if/elif/else`.

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (b) `elif`
- Q2. (b) Python raises a `SyntaxError`
- Q3. `A` then `B` (both `if` s are independent; both are True)
- Q4. `medium` (`n=7 > 5`, so first `elif` matches; rest skipped)
- Q5. Indent the print: `print("yes")` (4 spaces)
- Q6. Should be `if marks ≥ 40:` — `=` is assignment, `≥` is comparison

CHAPTER 07

Control Flow — Loops & Iteration

Decisions let your program choose between paths. Loops let your program *repeat* a path. Together, decisions and loops are the two pillars of all programming — with

them, you can write any program. Without them, you can write only the simplest scripts.

7.1 Why loops?

Imagine your teacher asks you to write "I will pay attention in class" on the blackboard 100 times. Doing it by hand is tedious. Asking a computer to do it without loops would also be tedious — you would have to write `print("I will pay attention")` 100 times in your code. Loops solve this elegantly: you write the instruction once, and tell Python how many times to repeat it.

Python has two main types of loops: the `for` loop and the `while` loop. Each is best for a different situation.

7.2 The `for` loop

A `for` loop is used when you know in advance how many times you want to repeat something, or when you want to iterate over a sequence (like a list of items, a string of characters, or a range of numbers).

The `range()` function

The most common companion of the `for` loop is the `range()` function, which generates a sequence of numbers.

Call	Produces
<code>range(5)</code>	0, 1, 2, 3, 4 (starts at 0, stops before 5)
<code>range(2, 6)</code>	2, 3, 4, 5 (starts at 2, stops before 6)
<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9 (starts at 1, steps by 2)
<code>range(10, 0, -1)</code>	10, 9, 8, ..., 1 (counts down)

— REMEMBER — RANGE STOPS BEFORE THE END VALUE

This is the single most common beginner mistake with `range`. `range(1, 5)` gives `1, 2, 3, 4` — *not* `1, 2, 3, 4, 5`. The stop value is exclusive. If you want to count from 1 to 10 inclusive, use `range(1, 11)`.

Syntax and examples

```
# Print "Hello" 3 times
for i in range(3):
    print("Hello")
```

OUTPUT

```
Hello  
Hello  
Hello
```

```
# Print numbers 1 to 5  
for num in range(1, 6):  
    print(num)
```

OUTPUT

```
1  
2  
3  
4  
5
```

```
# Print the squares of numbers 1 to 5  
for i in range(1, 6):  
    print(f"{i} squared is {i ** 2}")
```

OUTPUT

```
1 squared is 1  
2 squared is 4  
3 squared is 9  
4 squared is 16  
5 squared is 25
```

```
# Iterate over a string (each character)  
for char in "Python":  
    print(char)
```

OUTPUT

```
P  
y  
t  
h  
o  
n
```

7.3 The `while` loop

A `while` loop repeats as long as a condition is `True`. You use it when you do not know in advance exactly how many iterations you need — you just want to keep going until some condition becomes `False`.

Syntax

```
while condition:
    # code to repeat
    statement_1
    statement_2
    # something must change the condition, or loop runs forever!
```

Example — countdown

```
count = 5
while count > 0:
    print(count)
    count -= 1      # without this line, the loop runs forever
print("Lift off!")
```

OUTPUT

```
5
4
3
2
1
Lift off!
```

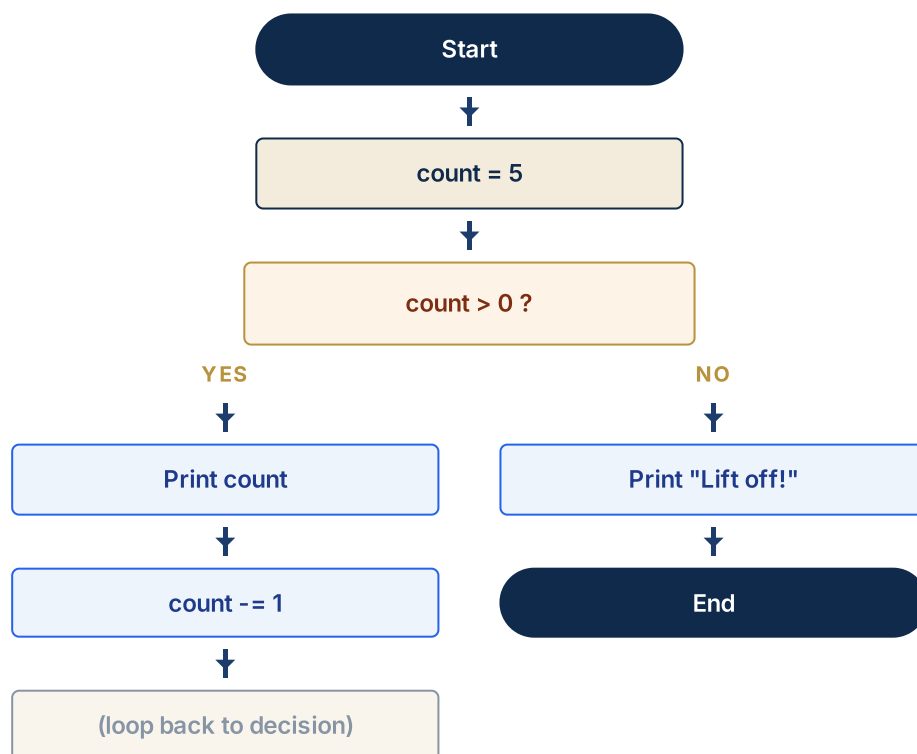


FIGURE 7.1 — FLOWCHART OF A **WHILE** LOOP

7.4 `for` vs `while` — when to use which

Aspect	<code>for</code> loop	<code>while</code> loop
When to use	When you know the number of iterations in advance	When you don't know the count — you loop until a condition changes
Typical example	Print numbers 1 to 100	Keep asking the user for input until they type "quit"
Counter	Provided by <code>range()</code>	You must manage the counter yourself
Infinite loop risk	Low — <code>range</code> always ends	High — if you forget to update the condition
Readability	Often cleaner for fixed counts	Cleaner for "keep going until..." logic

7.5 `break` and `continue`

Sometimes you want to exit a loop early, or skip an iteration. Python provides two keywords for this.

`break` — immediately exits the loop, skipping any remaining iterations.

```
# Find the first number divisible by 7 between 1 and 100
for n in range(1, 101):
    if n % 7 == 0:
        print(f"Found: {n}")
        break
print("Done.")
```

OUTPUT

```
Found: 7
Done.
```

`continue` — skips the rest of the current iteration and jumps to the next one.

```
# Print numbers 1 to 10, but skip the multiples of 3
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i)
```


OUTPUT

```
1
2
4
5
7
8
10
```

7.6 Nested loops

You can put a loop inside another loop. This is common when working with tables, grids, or 2D data. A classic example is the multiplication table.

```
# Multiplication table 1 to 5
for i in range(1, 6):
    for j in range(1, 6):
        print(i * j, end="\t")
    print()    # newline after each row
```

OUTPUT

```
1  2  3  4  5
2  4  6  8  10
3  6  9  12 15
4  8  12 16 20
5  10 15 20 25
```

The outer loop iterates over rows, the inner loop over columns. The inner loop runs completely for each iteration of the outer loop. Be careful — nested loops can be slow if the counts are large (a 100×100 nested loop runs 10,000 times).

7.7 Infinite loops and how to escape them

A `while` loop whose condition never becomes `False` is an **infinite loop**. The program will keep running forever (or until the computer runs out of memory). To stop a runaway program in the terminal, press `Ctrl + C`.

```
# DANGER: infinite loop!
count = 1
while count > 0:    # count is always > 0, never becomes False
    print(count)
    count += 1      # we increase it, but the condition stays True
```

— COMMON MISTAKE — FORGETTING TO UPDATE THE LOOP VARIABLE

```
count = 5
while count > 0:
    print(count)    # count never changes – infinite loop!
```

Fix: always include a statement inside the loop that moves the condition toward False. Here, add `count -= 1` inside the loop.

Practice Questions

MULTIPLE CHOICE

Q1. What does `range(3, 10, 2)` produce?

- A. 3, 5, 7, 9, 10
- B. 3, 5, 7, 9
- C. 3, 10, 2
- D. 3, 4, 5, 6, 7, 8, 9

Q2. Which keyword skips the rest of the current iteration and moves to the next?

- A. `break`
- B. `skip`
- C. `continue`
- D. `next`

OUTPUT PREDICTION

Q3. What is printed?

```
for i in range(5):  
    print(i * 2)
```

Q4. What is printed?

```
for i in range(1, 6):  
    if i == 3:  
        break  
    print(i)
```

Q5. What is printed?

```
n = 0  
while n < 10:  
    n += 3  
print(n)
```

TRUE / FALSE

Q6. A `for` loop can be used to iterate over a string character by character.

Q7. A `while` loop whose condition never becomes False will run forever unless interrupted.

DEBUGGING

Q8. This loop was meant to print numbers 1 to 5, but it prints 0 to 4. Fix it:

```
for i in range(5):  
    print(i)
```

SHORT CODING

Q9. Write a program to print the sum of all numbers from 1 to 100. (Hint: start with `total = 0` and add each number using a `for` loop.)

Q10. Write a program that asks the user for a number and prints its multiplication table from 1 to 10.

— INTERVIEW CORNER

- Q1.** When would you use a `for` loop vs a `while` loop? Give one example of each.
- Q2.** What is an infinite loop? How do you stop a program stuck in one?
- Q3.** What is the difference between `break` and `continue` ?

— FUN FACT

In Python 3, `range()` does not actually create a list of numbers. It is a "lazy" object that generates each number on demand, one at a time. This means `range(1000000000)` takes essentially no memory — Python does not store a billion numbers, it just remembers "start at 0, stop at a billion, step by 1" and produces each value as the loop asks for it. This is one reason Python can handle very large ranges without crashing.

QUICK RECAP

- `for` loop: use when you know the number of iterations or iterate over a sequence.
- `range(n)` produces 0 to n-1. `range(a, b)` produces a to b-1. `range(a, b, step)` uses a custom step.
- `while` loop: use when you loop until a condition becomes False.
- Always update the loop variable inside a `while` loop, or it runs forever.
- `break` exits the loop entirely. `continue` skips to the next iteration.
- Loops can be nested; common for tables and 2D data.
- Press `Ctrl + C` to stop a runaway program.

HOMEWORK

1. Write a program that prints all even numbers from 1 to 50.
2. Write a program that prints the factorial of a number entered by the user. (Factorial of 5 = $5 \times 4 \times 3 \times 2 \times 1 = 120$.)
3. Write a program that keeps asking the user to enter a number until they type 0, then prints the sum of all entered numbers.
4. Write a program that prints the following pattern using nested loops:

OUTPUT

```
*
**
***
****
*****
```

5. Write a program that checks if a number entered by the user is prime. (A prime number is divisible only by 1 and itself. Use a loop to check divisibility from 2 to n-1.)

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (b) 3, 5, 7, 9 — stop value 10 is exclusive
- Q2. (c) `continue`
- Q3. `0 2 4 6 8` (each on its own line)
- Q4. `1 2` (loop breaks at `i=3`, before printing)
- Q5. `12` — `n` goes 3, 6, 9, 12; at `n=12` the condition `n<10` is False
- Q6. True
- Q7. True
- Q8. Use `range(1, 6)` instead of `range(5)`

CHAPTER 08 • CHAPTER PROJECT 2

Number Guessing Game

Chapter Project 2

A Number Guessing Game in Python

Bringing together conditional statements, loops, and a tiny new tool — Python's random module.

In this project you will build a game where the computer picks a secret number between 1 and 100, and the player has to guess it. After each guess, the computer tells the player whether their guess was too high, too low, or correct. The game ends when the player guesses correctly. This project uses everything from the last two chapters: `if/elif/else` for decisions, a `while` loop for repeated guesses, and a comparison to detect the win.

Project introduction

Number guessing games are a classic beginner project for a reason. They are small enough to write in a single sitting, fun enough to actually play, and they exercise every concept from the last two chapters. By the end of this project, you will have written your first interactive game — a program that responds differently each time, depending on what the player types.

Objective

Build a Python program that:

1. Picks a random integer between 1 and 100 (inclusive) without showing it.
2. Repeatedly asks the player to guess the number.
3. After each guess, prints "Too high!", "Too low!", or "Correct!".

4. Counts the number of attempts.
5. When the player guesses correctly, prints the number of attempts and ends.

Concepts used

CONCEPTS

Variables & input

CONCEPTS

if / elif / else

CONCEPTS

while loop

CONCEPTS

Comparison operators

CONCEPTS

import (modules)

CONCEPTS

Counter variable

A tiny new tool — the `random` module

To pick a random number, we use a Python **module** called `random`. A module is a collection of pre-written code that you can use in your own programs. We will study modules in detail in Chapter 10 — for now, you just need two facts:

1. The line `import random` at the top of your file makes the `random` module available.
2. The function call `random.randint(1, 100)` returns a random integer between 1 and 100 (inclusive, unlike `range`).

```
import random
secret = random.randint(1, 100)
print(secret)    # run this multiple times – you'll get a different number
                  each time
```

Algorithm — the complete thought process

STEP-BY-STEP PLAN

- 1 Import the `random` module.
- 2 Pick a random integer between 1 and 100 and store it in a variable called `secret`.
- 3 Create a counter variable `attempts = 0` to track how many guesses the player has made.
- 4 Use a `while True` loop (an infinite loop that we will exit using `break` when the player wins).
- 5 Inside the loop, ask the player for a guess and convert it to an integer.
- 6 Increase `attempts` by 1.
- 7 Compare the guess to `secret` using `if/elif/else`.
- 8 If the guess equals the secret, print a win message with the attempt count, then `break` out of the loop.
- 9 If the guess is greater, print "Too high!".
- 10 If the guess is smaller, print "Too low!".

Notice how the thought process comes *before* the code. A professional programmer does not start typing until they can describe the algorithm in plain English. Code is just the translation of that plan into a language the computer understands.

The complete Python code

```
# guessing_game.py - a number guessing game
import random

# 1. Pick a random secret number
secret = random.randint(1, 100)
attempts = 0

print("Welcome to the Number Guessing Game!")
print("I'm thinking of a number between 1 and 100.")

# 2. Loop until the player guesses correctly
while True:
    guess = int(input("\nEnter your guess: "))
    attempts += 1

    # 3. Compare guess to secret
    if guess == secret:
        print(f"Correct! You guessed it in {attempts} attempts.")
        break
    elif guess < secret:
        print("Too low! Try a higher number.")
    else:
        print("Too high! Try a lower number.")
```

Code explanation

Line 2 — `import random`: This brings Python's random number tools into our program. Without it, the `random.randint` function would not exist.

Line 5 — picking the secret: `random.randint(1, 100)` returns a random integer between 1 and 100 inclusive. Note that `randint` includes both endpoints, unlike `range` which is exclusive of the stop value.

Line 6 — attempts counter: We start at 0 and add 1 each time the player guesses. This lets us report the total number of attempts at the end.

Line 11 — `while True:`: This is an infinite loop by design — we will exit it with `break` when the player wins. This is a common Python pattern when you want to loop an unknown number of times until something specific happens.

Lines 12–13 — reading a guess: We read the player's guess as a string, then immediately convert it to an integer with `int()`. We then increment the attempts counter.

Lines 16–22 — the comparison: Three branches. If equal, we print the win message and `break` out of the loop. If the guess is too low, we tell the player. If too high, we tell them that too. The loop then repeats and asks for another guess.

Sample input & output

SAMPLE SESSION

Welcome to the Number Guessing Game!

I'm thinking of a number between 1 and 100.

> Enter your guess: 50

Too low! Try a higher number.

> Enter your guess: 75

Too high! Try a lower number.

> Enter your guess: 60

Too low! Try a higher number.

> Enter your guess: 68

Correct! You guessed it in 4 attempts.

Possible improvements

- **Limit the number of attempts:** Give the player a maximum of 7 attempts (with the right strategy, you can always guess a number from 1 to 100 in 7 tries — this is a famous result called binary search).
- **Hint at how close they are:** Instead of just "Too high" or "Too low", print "Very close!" if the guess is within 5 of the secret.
- **Score system:** Award more points for fewer attempts. Display the score at the end.
- **Play again:** After a win, ask the player "Play again? (y/n)" and start a new game if they say yes.
- **Difficulty levels:** Let the player choose Easy (1–50), Medium (1–100), or Hard (1–500).

— PRACTICE CHALLENGE

Modify the game so that the player has a maximum of 7 attempts. If they fail to guess in 7 attempts, print `"Game over! The number was X."` and end the game. (Hint: change the `while True` to `while attempts < 7`, and after the loop check whether they won or ran out of attempts.)

— PROJECT REFLECTION

This project shows how decisions and loops combine to make programs interactive. The `while` loop keeps the game alive across multiple guesses, the `if/elif/else` decides what feedback to give, and the `break` exits cleanly when the game ends. These three pieces — loop, decision, exit — form the backbone of nearly every interactive program you will ever write.

Strings — Python's Text Data Structure

Almost every program deals with text at some point — a user's name, an email address, a sentence, a paragraph. Python stores text in a data type called a **string**. Strings are so important that they get their own chapter. By the end of this chapter, you will be able to slice, search, transform and combine text with confidence.

9.1 What is a string?

A string is an ordered sequence of characters enclosed in quotes. Python accepts three kinds of quotes:

```
# Single quotes
name = 'Aisha'

# Double quotes – identical to single quotes
greeting = "Hello, World!"

# Triple quotes – for multi-line strings
paragraph = """This is a longer string
that spans multiple lines.
Useful for docstrings and long text."""

# Empty string
empty = ""
```

Single and double quotes are interchangeable — pick whichever you find more readable. A common convention is to use double quotes when the text contains an apostrophe (`"It's a lovely day"`) and single quotes otherwise.

9.2 String indexing — accessing individual characters

Each character in a string has a numbered position called an **index**, starting from 0. You can access any character using square brackets `[ ]`.

P	y	t	h	o	n
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

FIGURE 9.1 — INDEX POSITIONS FOR THE STRING "PYTHON" (POSITIVE INDICES 0–5, NEGATIVE INDICES -6 TO -1)

```
word = "Python"
print(word[0])    # P (first character)
print(word[1])    # y
print(word[5])    # n (last character)

# Negative indices count from the end
print(word[-1])   # n (last character)
print(word[-2])   # o (second from last)
```

OUTPUT

```
P
y
n
n
o
```

— COMMON MISTAKE — INDEX OUT OF RANGE

```
word = "Python"
print(word[6])    # IndexError: string index out of range
```

Why: "Python" has 6 characters indexed 0–5. There is no index 6. **Fix:** use `len(word)` to find the length first, and remember the last valid index is `len - 1`.

9.3 String slicing — extracting parts

Slicing lets you extract a substring (a part of a string) using the syntax `[start:stop:step]`. As with `range`, the `stop` index is exclusive.

```
word = "Python"
```

```
print(word[0:3])    # Pyt    (indices 0, 1, 2)
print(word[2:5])    # tho
print(word[:3])     # Pyt    (omitted start defaults to 0)
print(word[3:])     # hon    (omitted stop defaults to end)
print(word[:])      # Python (whole string)
print(word[::2])    # Pto    (every 2nd character)
print(word[::-1])   # nohtyP (reversed!)
```

— TIP — THE REVERSE TRICK

`word[::-1]` is the Pythonic way to reverse a string. The step of `-1` tells Python to walk backwards from end to start. This idiom appears constantly in real code.

9.4 String immutability

Strings in Python are **immutable** — once created, they cannot be changed. You cannot do `word[0] = "J"` to turn "Python" into "Jython". If you want a modified version, you create a new string.

```
word = "Python"
# word[0] = "J"    # TypeError: strings are immutable

# Instead, build a new string
new_word = "J" + word[1:]
print(new_word)    # Jython
```

This may seem restrictive, but it has a major benefit: it makes strings safe to share. When you pass a string to a function, you can be sure the function cannot modify the original — it can only create new strings. This eliminates a whole class of bugs.

9.5 Common string methods

A **method** is a function that belongs to a particular object. Strings come with dozens of built-in methods. Here are the most useful ones for beginners.

Method	What it does	Example
<code>.upper()</code>	Returns uppercase copy	<code>"hi".upper()</code> → <code>"HI"</code>
<code>.lower()</code>	Returns lowercase copy	<code>"HI".lower()</code> → <code>"hi"</code>
<code>.strip()</code>	Removes leading/trailing whitespace	<code>" hi ".strip()</code> → <code>"hi"</code>
<code>.replace(a, b)</code>	Replaces all occurrences of a with b	<code>"cat".replace("c", "b")</code> → <code>"bat"</code>
<code>.find(sub)</code>	Returns index of first occurrence, or -1	<code>"hello".find("l")</code> → <code>2</code>
<code>.split(sep)</code>	Splits into a list of substrings	<code>"a,b,c".split(",")</code> → <code>['a', 'b', 'c']</code>
<code>.startswith(s)</code>	True if string starts with s	<code>"Hello".startswith("He")</code> → <code>True</code>
<code>.endswith(s)</code>	True if string ends with s	<code>"file.py".endswith(".py")</code> → <code>True</code>
<code>len(s)</code>	Returns length (function, not method)	<code>len("Python")</code> → <code>6</code>

```

email = "  Aisha@Example.COM  "

# Clean up: strip spaces, lowercase
clean_email = email.strip().lower()
print(clean_email)    # aisha@example.com

# Check if it looks like an email
if "@" in clean_email and clean_email.endswith(".com"):
    print("Valid format.")

# Replace parts
new_email = clean_email.replace("example", "gmail")
print(new_email)    # aisha@gmail.com

```

9.6 Concatenation and repetition

The `+` operator joins two strings (concatenation). The `*` operator repeats a string a given number of times.

```
first = "Hello"
second = "World"

combined = first + ", " + second + "!"
print(combined)      # Hello, World!

print("-" * 30)      # -----
print("Ha" * 3)       # HaHaHa
```

— REMEMBER

- Strings are immutable — methods return *new* strings; they do not change the original.
- Indexing starts at 0; negative indices count from the end.
- Slicing: `[start:stop:step]`, stop is exclusive.
- Use `+` to concatenate, `*` to repeat.
- Always call string methods on the string itself with a dot: `s.upper()`, not `upper(s)`.

Practice Questions

OUTPUT PREDICTION

Q1. What is printed?

```
s = "Hello"
print(s[1:4])
```

Q2. What is printed?

```
print("Python"[::-1])
```

Q3. What is printed?

```
name = " aisha "
print(name.strip().upper())
```

Q4. What is printed?

```
print("a-b-c-d".split("-"))
```

MULTIPLE CHOICE

Q5. Strings in Python are:

- A. Mutable — you can change individual characters
- B. Immutable — you cannot change individual characters
- C. Sometimes mutable, sometimes not
- D. Mutable only inside functions

DEBUGGING

Q6. This code is meant to capitalize the first letter of `word`, but it raises an error. Explain why and fix it:

```
word = "python"  
word[0] = "P"
```

SHORT CODING

Q7. Write a program that asks the user for a sentence and prints (a) the number of characters in it, (b) the sentence in uppercase, and (c) the sentence reversed.

Q8. Write a program that asks the user for their full name and prints just their first name (assume a single space separates first and last name).

— INTERVIEW CORNER

Q1. What does it mean to say strings are immutable? Why is this a useful property?

Q2. Explain the difference between `s.find("x")` and `"x" in s`. What does each return?

— FUN FACT

Python strings are **Unicode** — they can hold text in any language on Earth. You can write `greeting = "नमस्ते"` or `greeting = "你好"` or even mix languages and emoji in a single string. The length of `"नमस्ते"` is 6, not 5 — because some Indian characters are composed of multiple code points. This is one reason Python is so popular for international text processing.

QUICK RECAP

- Strings are ordered sequences of characters, written in single, double, or triple quotes.
- Indexing: `s[i]` — 0-based, supports negative indices from the end.
- Slicing: `s[start:stop:step]` — stop is exclusive.
- Strings are immutable — methods return new strings, never modify the original.
- Useful methods: `.upper()`, `.lower()`, `.strip()`, `.replace()`, `.find()`, `.split()`, `.startswith()`, `.endswith()`, `len()`.
- `+` concatenates, `*` repeats.
- `s[::-1]` reverses a string.

HOMEWORK

1. Write a program that checks if a word entered by the user is a palindrome (reads the same forwards and backwards, like "madam").
2. Write a program that asks the user for an email address and prints whether it is valid (contains exactly one `@`, and ends with `.com` or `.in`).
3. Write a program that counts how many vowels (a, e, i, o, u) are in a sentence entered by the user.
4. Write a program that asks the user for a sentence and prints the number of words in it (hint: split by spaces).
5. Write a program that takes a string and returns a new string where every lowercase vowel is replaced with `*`. So `"Hello World"` becomes `"H*ll* W*rld"`.

ANSWER KEY — PRACTICE QUESTIONS

- Q1. `ell` — indices 1, 2, 3
- Q2. `nohtyP`
- Q3. `AISHA`
- Q4. `['a', 'b', 'c', 'd']`
- Q5. (b) Immutable
- Q6. Strings are immutable; you cannot assign to an index. Fix: `word = "P" + word[1:]`

CHAPTER 10

Lists, Tuples, Sets & Dictionaries

Variables hold one value at a time. But real programs often need to handle many values: a list of student names, a set of unique tags, a dictionary of product prices. Python provides four built-in **data structures** for this — lists, tuples, sets, and

dictionaries. Each is designed for a different purpose. This chapter covers all four and shows you when to use which.

10.1 Why we need collections

Imagine you are building a phone book app. You need to store 50 contact names. Creating 50 separate variables (`name1` , `name2` , ..., `name50`) would be absurd. What if the user adds a 51st contact? You would have to write new code. The right solution is a single variable that can hold many values — a **collection**. Python's four core collections each have a different personality, designed for different situations.

10.2 Lists — ordered and mutable

A **list** is an ordered, changeable collection of items. Lists are the most flexible and most used collection in Python.

```
# Creating lists
fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", 3.14, True]    # can hold any types
empty = []
```

Indexing and slicing — same as strings

```
fruits = ["apple", "banana", "cherry", "date"]

print(fruits[0])        # apple
print(fruits[-1])       # date
print(fruits[1:3])      # ['banana', 'cherry']
print(len(fruits))      # 4
```



FIGURE 10.1 — A LIST IS AN ORDERED SEQUENCE OF SLOTS, EACH HOLDING ONE VALUE

List methods

Method	What it does
<code>.append(x)</code>	Adds <code>x</code> to the end of the list
<code>.insert(i, x)</code>	Inserts <code>x</code> at index <code>i</code>
<code>.remove(x)</code>	Removes the first occurrence of <code>x</code>
<code>.pop(i)</code>	Removes and returns the item at index <code>i</code> (last by default)
<code>.sort()</code>	Sorts the list in place (ascending)
<code>.reverse()</code>	Reverses the list in place
<code>.extend(other)</code>	Appends all items from another list
<code>x in list</code>	True if <code>x</code> is in the list

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")
print(fruits)           # ['apple', 'banana', 'cherry', 'date']

fruits.insert(1, "blueberry")
print(fruits)           # ['apple', 'blueberry', 'banana', 'cherry', 'date']

fruits.remove("banana")
print(fruits)           # ['apple', 'blueberry', 'cherry', 'date']

last = fruits.pop()
print(last, fruits)     # date ['apple', 'blueberry', 'cherry']

fruits.sort()
print(fruits)           # ['apple', 'blueberry', 'cherry']
```

10.3 Tuples — ordered and immutable

A **tuple** is like a list, but *immutable* — once created, it cannot be changed. Use a tuple when you have a fixed collection of related values that should never change, like the coordinates of a point or the days of the week.

```
# Creating tuples
point = (3, 4)
days = ("Mon", "Tue", "Wed", "Thu", "Fri")
single = (42,)      # comma makes it a tuple, not just a number in
                    # parentheses

print(point[0])      # 3
print(len(days))     # 5

# point[0] = 5      # TypeError: tuples are immutable
```

Tuples support indexing and slicing just like lists, but they do not support methods that modify them (no `.append`, `.remove`, etc.). They only have `.count(x)` (count occurrences of x) and `.index(x)` (find index of x).

10.4 Sets — unordered, unique elements

A **set** is an unordered collection of *unique* items. If you add the same item twice, the second add is ignored. Sets are perfect for removing duplicates from a list, or for mathematical operations like union and intersection.

```
# Creating sets
numbers = {1, 2, 3, 4, 5}
empty_set = set()    # NOT {} (that's an empty dict)

# Adding and removing
numbers.add(6)
numbers.add(3)        # 3 is already there; no effect
print(numbers)        # {1, 2, 3, 4, 5, 6}

numbers.discard(2)
print(numbers)        # {1, 3, 4, 5, 6}

# Removing duplicates from a list
names = ["Aisha", "Ravi", "Aisha", "Priya", "Ravi"]
unique_names = set(names)
print(unique_names)   # {'Aisha', 'Ravi', 'Priya'}

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)          # union: {1, 2, 3, 4, 5, 6}
print(a & b)          # intersection: {3, 4}
print(a - b)          # difference: {1, 2}
```

Note that sets are unordered — you cannot index into them with `s[0]`. They are also unhashable for the same reason. Use a set when you care about *membership* ("is X in this collection?") and *uniqueness*, not about order.

10.5 Dictionaries — key-value pairs

A **dictionary** stores data as *key-value pairs*. Each key is unique, and each key maps to exactly one value. Dictionaries are perfect for representing things like a phone book (name → number), a price list (item → price), or a configuration (option → setting).

KEY	VALUE
"name"	"Aisha"
"age"	21
"course"	"Python"

FIGURE 10.2 — A DICTIONARY MAPS UNIQUE KEYS TO VALUES

```
# Creating a dictionary
student = {
    "name": "Aisha",
    "age": 21,
    "course": "Python"
}

# Accessing values by key
print(student["name"])    # Aisha
print(student["age"])     # 21

# Adding or updating a value
student["grade"] = "A"    # adds a new key
student["age"] = 22       # updates an existing key

# Safe access with .get() – returns None instead of crashing
print(student.get("phone"))    # None (key doesn't exist)
print(student.get("phone", "not set")) # not set (default)

# Dictionary methods
print(student.keys())          # dict_keys(['name', 'age', 'course', 'grade'])
print(student.values())        # dict_values(['Aisha', 22, 'Python', 'A'])
print(student.items())         # dict_items([('name', 'Aisha'), ...])

# Deleting a key
del student["course"]
print(student)
```

10.6 Comparison of all four collections

Feature	List	Tuple	Set	Dictionary
Syntax	<code>[1, 2, 3]</code>	<code>(1, 2, 3)</code>	<code>{1, 2, 3}</code>	<code>{"k": "v"}</code>
Ordered?	Yes	Yes	No	Yes (Python 3.7+)
Mutable?	Yes	No	Yes	Yes
Duplicates?	Yes	Yes	No (unique)	Keys unique, values can repeat
Index access?	Yes <code>l[i]</code>	Yes <code>t[i]</code>	No	By key <code>d[k]</code>
Best for	Ordered, changeable list of items	Fixed records (coords, dates)	Unique items, membership checks	Lookup by key (phonebook, config)
Example use	Shopping cart items	GPS coordinates <code>(lat, lon)</code>	Set of tags	User profile by <code>user_id</code>

10.7 Choosing the right structure

— DECISION GUIDE

- Need ordered, changeable items? → Use a **list**.
- Need ordered, fixed items that should never change? → Use a **tuple**.
- Need unique items or fast membership tests? → Use a **set**.
- Need to look up values by a key? → Use a **dictionary**.

If unsure, start with a list — it is the most flexible and most common. Refactor later if you find a better fit.

— COMMON MISTAKES

- Using `{}` for an empty set — this creates an empty *dictionary*. Use `set()` for an empty set.
- Trying to modify a tuple — tuples are immutable; use a list if you need to change items.
- Indexing a set — sets are unordered; you cannot do `s[0]`.
- Using a list as a dictionary key — keys must be immutable. Use a tuple instead.
- Accessing a missing dictionary key with `d[key]` — this raises `KeyError`. Use `d.get(key)` for safety.

Practice Questions

MULTIPLE CHOICE

Q1. Which collection does NOT allow duplicate values?

- A. List
- B. Tuple
- C. Set
- D. Dictionary values

Q2. Which collection is immutable?

- A. List
- B. Tuple
- C. Set
- D. Dictionary

Q3. What does `len({})` return?

- A. 0
- B. 1
- C. An error
- D. None

OUTPUT PREDICTION

Q4. What is printed?

```
nums = [3, 1, 4, 1, 5]
print(sorted(set(nums)))
```

Q5. What is printed?

```
d = {"a": 1, "b": 2}
d["c"] = 3
print(list(d.keys()))
```

Q6. What is printed?

```
t = (1, 2, 3)
print(t[1])
```

TRUE / FALSE

Q7. You can store a list as a value inside a dictionary.

Q8. Tuples support `.append()` and `.remove()` methods.

DEBUGGING

Q9. This code raises a `KeyError`. Fix it without changing the logic:

```
config = {"host": "localhost", "port": 8080}  
print(config["user"])
```

SHORT CODING

Q10. Write a program that creates a list of 5 numbers, then prints (a) the sum, (b) the maximum, (c) the minimum, and (d) the average. (Hint: use the built-in `sum()`, `max()`, `min()` functions.)

Q11. Write a program that asks the user for 5 names, stores them in a list, then prints them sorted alphabetically.

Q12. Write a program that creates a dictionary mapping fruit names to their prices, then asks the user for a fruit name and prints its price (or "Not found" if missing).

— INTERVIEW CORNER

Q1. What is the difference between a list and a tuple? When would you choose one over the other?

Q2. What is the difference between a set and a dictionary?

Q3. Why can a tuple be used as a dictionary key but a list cannot?

Q4. What does `dict.get(key, default)` do, and why is it safer than `dict[key]` ?

— FUN FACT

Since Python 3.7, dictionaries preserve insertion order — the order in which you add keys is the order in which they appear when you iterate. Before 3.7, the order was arbitrary. This was a major change driven by how often real code relies on order, and it has made dictionaries much more pleasant to use. (In Python 3.6 it was an implementation detail; from 3.7 it is a guaranteed language feature.)

QUICK RECAP

- **List** `[ ]` — ordered, mutable, allows duplicates. Methods: `append`, `insert`, `remove`, `pop`, `sort`, `reverse`.
- **Tuple** `( )` — ordered, immutable. Use for fixed records.
- **Set** `{ }` — unordered, unique. Use for membership and de-duplication. Empty set: `set()`.
- **Dictionary** `{k: v}` — key-value pairs, mutable. Use `.get()` for safe access.
- Choose by need: ordered+mutable → list; fixed → tuple; unique → set; lookup by key → dict.

HOMework

1. Write a program that asks the user for 10 numbers, stores them in a list, and prints the largest and smallest numbers (without using `max` / `min`).
2. Write a program that takes a list of names with duplicates, removes the duplicates using a set, and prints the unique names sorted alphabetically.
3. Write a program that maintains a dictionary of student names and their marks. Allow the user to add 3 students, then print the dictionary sorted by marks (highest first).
4. Write a program that counts how many times each word appears in a sentence entered by the user. Store the counts in a dictionary.
5. Create a list of 5 tuples, where each tuple contains a student's name and their age. Print the names of all students who are 18 or older.

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (c) Set
- Q2. (b) Tuple
- Q3. (a) 0 — `{}` is an empty dict, `len` is 0
- Q4. `[1, 3, 4, 5]` — set removes duplicates, sorted sorts
- Q5. `['a', 'b', 'c']`
- Q6. `2`
- Q7. True
- Q8. False — tuples are immutable
- Q9. Use `config.get("user")` or `config.get("user", "default")` — no `KeyError`

Student Management Program

Chapter Project 3

A CLI Student Management System

Combining lists, dictionaries, control flow and loops into a single useful application.

In this project you will build a small command-line application for managing students. The user will be able to add a student, view all students, search by name, and delete a student — all from a menu. This is your first taste of building a "real" application: one that does more than a single calculation, has a user interface (even if it is text-based), and lets the user choose what to do.

Project introduction

So far your programs have done one thing each: a calculator calculates, a guessing game guesses. Real applications, however, offer a menu of related actions and let the user pick. The Student Management Program is the smallest example of this pattern — a **menu-driven CLI application**. Once you can build this, you can build any small command-line tool.

Objective

Build a Python program that:

1. Stores a list of students, where each student is a dictionary with name, roll number, and grade.
2. Displays a menu: 1. Add student 2. View all 3. Search by name 4. Delete by roll 5. Exit
3. Loops until the user picks Exit.
4. Each menu option performs its task using list and dictionary operations.

Concepts used

CONCEPTS

List of dicts

CONCEPTS

while loop + break

CONCEPTS

if / elif / else

CONCEPTS

input() & print()

CONCEPTS

String .lower() for menu

CONCEPTS

for loop (for search/view)

Algorithm — the logical approach

STEP-BY-STEP PLAN

- 1 Create an empty list called `students`. Each item will be a dictionary.
- 2 Start an infinite loop (a menu loop).
- 3 Display the menu and read the user's choice.
- 4 Use `if/elif/else` to handle each choice:
 - 5 **Add:** Ask for name, roll, grade. Create a dictionary and `.append()` it to the list.
 - 6 **View all:** Loop through the list and print each student's details.
 - 7 **Search by name:** Loop through the list; if the name matches, print the student.
 - 8 **Delete by roll:** Loop through the list; if the roll matches, remove that student.
 - 9 **Exit:** Break out of the loop.
- 10 Handle invalid menu choices gracefully.

The complete Python code

```
# student_management.py - a CLI student management system

students = [] # list of dictionaries, each holding a student

while True:
    print("\n=== Student Management Menu ===")
    print("1. Add student")
    print("2. View all students")
    print("3. Search by name")
    print("4. Delete by roll number")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == "1":
        name = input("  Name: ")
        roll = input("  Roll number: ")
        grade = input("  Grade: ")
        student = {"name": name, "roll": roll, "grade": grade}
        students.append(student)
        print(f"  Added: {name}.")

    elif choice == "2":
        if not students:
            print("  No students yet.")
        else:
            print("\n  -- All Students --")
            for s in students:
                print(f"  {s['roll']} | {s['name']} | Grade: {s['grade']}")

    elif choice == "3":
        search = input("  Name to search: ").lower()
        found = False
        for s in students:
            if search == s["name"].lower():
                print(f"  Found: {s['roll']} | {s['name']} | Grade: {s['grade']}")
                found = True
        if not found:
            print("  No matching student.")

    elif choice == "4":
        roll_to_delete = input("  Roll number to delete: ")
        for i, s in enumerate(students):
            if s["roll"] == roll_to_delete:
                del students[i]
                print(f"  Deleted student with roll {roll_to_delete}.")
                break
        else:
            print("  No student with that roll number.")
```

```
elif choice == "5":
    print("Goodbye!")
    break

else:
    print(" Invalid choice. Please pick 1-5.")
```

Code explanation

Line 3 — `students = []`: We start with an empty list. Each student we add will be a dictionary appended to this list. This combination — a list of dictionaries — is an extremely common pattern for representing collections of records.

Lines 5–11 — the menu loop: A `while True` loop displays the menu each time and reads the user's choice. We use strings "1" through "5" rather than integers because `input()` returns a string, and converting to int would crash if the user typed something non-numeric. Comparing strings avoids that crash.

Lines 13–19 — Add: Read three values, build a dictionary, append it. Notice we use the same variable names as dictionary keys, which makes the code easy to read.

Lines 21–27 — View all: Loop through the list with `for s in students` and print each student. The check `if not students` handles the empty-list case gracefully instead of printing nothing.

Lines 29–38 — Search by name: Loop through students, compare names case-insensitively (using `.lower()` on both sides). Use a `found` flag to track whether we matched anything; if not, print "No matching student".

Lines 40–50 — Delete by roll: This is the most interesting part. We use `enumerate(students)` to get both the index `i` and the student `s` in each iteration. When we find a matching roll number, we `del students[i]` and break. The `else` clause on the `for` loop is a special Python feature — it runs only if the loop completed without hitting `break`. So if no student matched, the "No student with that roll number" message prints.

Lines 52–55 — Exit & invalid choice: Standard exit with `break`, and an `else` clause that catches anything not 1–5.

Sample session

SAMPLE SESSION

≡ Student Management Menu ≡

1. Add student
2. View all students
3. Search by name
4. Delete by roll number
5. Exit

> Enter your choice (1-5): 1

Name: Aisha Khan

Roll number: CSE001

Grade: A

Added: Aisha Khan.

≡ Student Management Menu ≡

...

> Enter your choice (1-5): 1

Name: Ravi Patel

Roll number: CSE002

Grade: B

Added: Ravi Patel.

> Enter your choice (1-5): 2

-- All Students --

CSE001 | Aisha Khan | Grade: A

CSE002 | Ravi Patel | Grade: B

> Enter your choice (1-5): 3

Name to search: aisha

Found: CSE001 | Aisha Khan | Grade: A

> Enter your choice (1-5): 4

Roll number to delete: CSE002

Deleted student with roll CSE002.

> Enter your choice (1-5): 5

Goodbye!

Possible improvements

- **Persist data to a file:** Save the students list to a CSV file so the data survives when the program closes. (You will learn how in Chapter 13.)
- **Edit a student:** Add a menu option to edit an existing student's grade.
- **Input validation:** Check that the roll number is unique before adding a new student.
- **Sort by name or grade:** Add a "View sorted by name" option using `.sort(key= ...)`.
- **Find topper:** Add a menu option that finds the student with the highest grade.

— PRACTICE CHALLENGE

Add a "5. Find topper" menu option (renumber Exit to 6). The topper is the student with the highest grade. Assume grades are letters A–F where A is highest. Hint: create a dictionary mapping each grade letter to a numeric rank (`{"A": 5, "B": 4, "C": 3, "D": 2, "F": 1}`), then find the student whose grade has the highest rank.

— PROJECT REFLECTION

This project shows how a single program can do many things by combining a menu loop with `if/elif/else`. The same pattern — menu loop + decision tree — powers almost every command-line application you have ever used, from `git` to `pip`. You are now writing programs in the same shape as professional tools.

CHAPTER 12

Functions & Modular Programming

As your programs grow, you will find yourself writing the same logic in multiple places — asking the user for a number, validating input, computing a total. Repeating code is wasteful and error-prone: if you find a bug in one copy, you have to find and fix every other copy. **Functions** solve this problem by letting you wrap a piece of code in a name, and reuse it from anywhere in your program.

12.1 Why functions?

Imagine you are cooking, and your recipe says "boil water" three times. Instead of writing out the full instructions for boiling water each time, you would just say "boil water" and rely on the cook's knowledge. Functions are the programming equivalent of that — you define a step once, give it a name, and call it whenever you need it.

The benefits are enormous:

- **Reusability:** Write once, call many times.
- **Readability:** `calculate_tax(amount)` is clearer than a five-line formula inline.
- **Maintainability:** If the tax formula changes, you update it in one place.
- **Testability:** You can test a function in isolation, separately from the rest of the program.

This approach is called **modular programming** — breaking a large program into small, named, reusable modules (functions). Almost every professional Python program is organised this way.

12.2 Defining and calling a function

```
# Define a function
def greet():
    print("Hello, welcome to Python!")

# Call it
greet()
greet()    # call as many times as you like
```

OUTPUT

```
Hello, welcome to Python!
Hello, welcome to Python!
```

The `def` keyword tells Python you are defining a function. The name (here, `greet`) follows the same rules as variable names. The parentheses `()` can hold **parameters** (inputs) — more on those shortly. The colon and indented block work just like with `if` and `for`.

Note an important distinction: `def greet():` defines the function but does not run it. The code inside only runs when you *call* the function by writing `greet()`. A common beginner mistake is to define a function but forget to call it, then wonder why nothing happens.

12.3 Parameters and arguments

A **parameter** is a variable listed in the function definition. An **argument** is the actual value you pass when calling the function. The function uses the parameter name to refer to whatever argument was passed.

```
def greet(name):                # name is a parameter
    print(f"Hello, {name}!")

greet("Aisha")                  # "Aisha" is an argument
greet("Ravi")                   # "Ravi" is a different argument
greet("World")                  # "World" is yet another
```

OUTPUT

```
Hello, Aisha!
Hello, Ravi!
Hello, World!
```

A function can take multiple parameters, separated by commas:

```
def add(a, b):
    print(f"{a} + {b} = {a + b}")

add(3, 5)
add(10, 20)
```

OUTPUT

```
3 + 5 = 8
10 + 20 = 30
```

12.4 The `return` statement

So far, our functions have *printed* their result. But in real programs, you usually want a function to **return** a value that the caller can use. The `return` statement sends a value back to wherever the function was called.

```
def add(a, b):
    return a + b

result = add(3, 5)      # result is now 8
print(result)          # 8
print(add(10, 20))     # 30 - can use directly
print(add(1, 2) + add(3, 4)) # 3 + 7 = 10
```

— COMMON MISTAKE — FORGETTING `RETURN`

```
def double(x):
    x * 2      # computes but doesn't return!

result = double(5)
print(result) # None
```

Why: without `return`, a function returns the special value `None` by default. The computation `x * 2` happens but is thrown away. **Fix:** add `return x * 2`.

12.5 Multiple return values

Python functions can return multiple values (technically as a tuple, which Python conveniently unpacks):

```
def min_max(numbers):
    return min(numbers), max(numbers)

lowest, highest = min_max([4, 2, 9, 1, 7])
print(f"Min: {lowest}, Max: {highest}") # Min: 1, Max: 9
```

12.6 Default arguments

You can give a parameter a default value. If the caller does not provide an argument for that parameter, the default is used.

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Aisha")           # Hello, Aisha!
greet("Ravi", "Hi")      # Hi, Ravi!
greet("Sam", "Good morning") # Good morning, Sam!
```

Rule: parameters with defaults must come *after* parameters without defaults.

12.7 Keyword arguments

You can also pass arguments by name, in any order:

```
def describe(name, age, city):
    print(f"{name}, {age}, from {city}")

describe(city="Mumbai", name="Aisha", age=21)
# Aisha, 21, from Mumbai
```

12.8 Docstrings

A **docstring** is a triple-quoted string immediately after the `def` line that documents what the function does. It is a Python convention that every function should have one.

```
def area_of_circle(radius):
    """Return the area of a circle with the given radius."""
    return 3.14159 * radius ** 2
```

12.9 Variable scope — local vs global

Variables created inside a function are **local** — they only exist while the function is running. Variables created outside all functions are **global** — they can be read from anywhere, but modifying them from inside a function requires the `global` keyword (which is generally a bad idea — prefer passing values as arguments).

```
counter = 0    # global

def increment():
    local_var = 10    # local — only exists inside this function
    return local_var + 1

print(increment())    # 11
# print(local_var)    # NameError — local_var does not exist out here
```


REMEMBER

- `def` defines a function; calling it (with parentheses) runs it.
- Parameters go in the `def` line; arguments are passed when calling.
- `return` sends a value back. Without it, the function returns `None`.
- Default arguments make some parameters optional.
- Variables inside a function are local; they vanish when the function returns.
- Every function should have a docstring explaining what it does.

Practice Questions

OUTPUT PREDICTION

Q1. What is printed?

```
def mystery(x):  
    return x * 2  
  
print(mystery(5))
```

Q2. What is printed?

```
def f(x, y=10):  
    return x + y  
  
print(f(5))  
print(f(5, 20))
```

Q3. What is printed?

```
def g():  
    print("hi")  
  
result = g()  
print(result)
```

MULTIPLE CHOICE

Q4. What does a function return if it has no `return` statement?

- A. 0
- B. An empty string
- C. `None`
- D. It raises an error

DEBUGGING

Q5. This function was supposed to double a number, but the result is wrong. Fix it:

```
def double(x):  
    x * 2  
  
print(double(5))    # prints None
```

SHORT CODING

Q6. Write a function `is_even(n)` that returns `True` if `n` is even, `False` otherwise. Then write a loop that uses it to print all even numbers from 1 to 20.

Q7. Write a function `celsius_to_fahrenheit(c)` that returns the Fahrenheit equivalent. Then call it for `c = 0, 25, 100` and print the results.

— INTERVIEW CORNER

Q1. What is the difference between a parameter and an argument?

Q2. What does a function return if it has no `return` statement?

Q3. Explain the difference between local and global variables. Why is modifying globals from inside functions discouraged?

— FUN FACT

In Python, functions are **first-class objects**. That means you can assign a function to a variable, pass it as an argument to another function, or return it from a function. For example, you can write `f = print` and then call `f("hello")` — it works! This ability to treat functions as data is the foundation of functional programming, decorators, and many advanced Python features you will meet later.

QUICK RECAP

- Functions wrap reusable code: `def name(params):`
- Parameters are inputs in the definition; arguments are values passed when calling.
- `return` sends a value back. Without it, the function returns `None`.
- Default arguments: `def f(x, y=10):` — `y` is optional.
- Keyword arguments: `f(y=20, x=5)` — pass by name in any order.
- Local variables exist only inside the function. Globals are read-only from inside (without `global`).
- Use docstrings to document what each function does.

HOMework

1. Write a function `factorial(n)` that returns the factorial of `n` using a loop. Test it with `n = 5` (should be 120) and `n = 10`.
2. Write a function `is_prime(n)` that returns `True` if `n` is prime, `False` otherwise. Use it to print all primes from 1 to 50.
3. Write a function `reverse_string(s)` that returns the reversed version of `s`. (Do not use `s[::-1]` — build it yourself with a loop.)
4. Write a function `calculate_bmi(weight, height)` that returns the BMI and a category string ("Underweight", "Normal", "Overweight", "Obese").
5. Refactor the Calculator project (Chapter 5) into functions: `add(a,b)`, `subtract(a,b)`, etc. The main program should ask for two numbers and call the appropriate function.

ANSWER KEY — PRACTICE QUESTIONS

- Q1. `10`
- Q2. `15` then `25` (default used first time, override second time)
- Q3. `hi` then `None` (function has no return)
- Q4. (c) `None`
- Q5. Add `return`: `def double(x): return x * 2`

CHAPTER 13

Modules, Exceptions & File Handling

Real programs do not live entirely in a single file. They use code from other files (modules), handle errors gracefully (exceptions), and read and write data that persists between runs (file handling). This chapter covers all three — the tools that turn a script into an application.

13.1 What is a module?

A **module** is simply a Python file whose code can be used by another Python file. If you write a file called `utils.py` containing some useful functions, any other Python file in the same folder can `import utils` and use those functions. This lets you split a large program into logical pieces across multiple files.

Python also comes with a huge **standard library** — hundreds of pre-written modules for common tasks. You have already used one: `random`, in the Number Guessing Game.

13.2 The `import` statement

```
# Import an entire module
import math
print(math.pi)           # 3.141592653589793
print(math.sqrt(16))     # 4.0

# Import specific items from a module
from random import randint, choice
print(randint(1, 10))    # a random integer 1-10
print(choice(["red", "green", "blue"])) # a random element

# Import with an alias (a shorter name)
import datetime as dt
now = dt.datetime.now()
print(now)               # 2025-08-08 14:23:11.123456
```

13.3 Common standard library modules

Module	What it provides
<code>math</code>	Mathematical functions: <code>sqrt</code> , <code>sin</code> , <code>cos</code> , <code>pi</code> , <code>e</code> , <code>log</code> , etc.
<code>random</code>	Random numbers: <code>randint</code> , <code>random</code> , <code>choice</code> , <code>shuffle</code>
<code>datetime</code>	Working with dates and times: <code>datetime.now()</code> , <code>date.today()</code>
<code>os</code>	Operating system functions: file paths, environment variables, directories
<code>csv</code>	Reading and writing CSV files
<code>json</code>	Reading and writing JSON data (common in web APIs)
<code>statistics</code>	Mean, median, mode, standard deviation
<code>string</code>	Useful constants like <code>string.ascii_letters</code>

13.4 Creating your own module

Any `.py` file is a module. To create one, just save your functions in a file.

```
# File: mymath.py
def square(x):
    return x * x

def cube(x):
    return x * x * x
```

```
# File: main.py (in the same folder)
from mymath import square, cube

print(square(5))    # 25
print(cube(3))      # 27
```

13.5 Why exception handling matters

Programs fail. A user might type "abc" when you asked for a number. A file you tried to open might not exist. A network connection might drop. Without exception handling, any of these errors will crash your program instantly — the user sees an ugly traceback and loses their work.

Real programs handle errors gracefully. They anticipate what might go wrong, catch the error when it happens, and either recover or show a friendly message. Python's tool for this is the `try/except` statement.

13.6 The `try/except` statement

```
try:
    age = int(input("Enter your age: "))
    print(f"Next year you'll be {age + 1}.")
except ValueError:
    print("That's not a valid number. Please try again.")
```

OUTPUT

```
Enter your age: abc
That's not a valid number. Please try again.
```

Here is how it works. Python tries to run the code inside the `try` block. If a `ValueError` occurs (because `int("abc")` fails), Python immediately jumps to the `except` block and runs the recovery code. The program does not crash — it handles the error and continues.

13.7 Common exception types

Exception	When it occurs
<code>ValueError</code>	Right type, wrong value — e.g. <code>int("abc")</code>
<code>TypeError</code>	Wrong type — e.g. <code>"5" + 5</code>
<code>ZeroDivisionError</code>	Dividing by zero
<code>IndexError</code>	List index out of range
<code>KeyError</code>	Dictionary key not found
<code>FileNotFoundError</code>	Trying to open a file that doesn't exist
<code>NameError</code>	Using a variable that was never defined

try / except / else / finally

```
try:
    f = open("data.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("File not found. Creating a new one.")
    content = ""
else:
    f.close()    # only runs if no exception occurred
finally:
    print("Attempt complete.")    # always runs, exception or not
```

The `else` block runs only if no exception was raised. The `finally` block runs no matter what — useful for cleanup tasks like closing files or database connections.

— COMMON MISTAKE — CATCHING TOO BROADLY

```
try:
    # lots of code
    ...
except:
    print("Something went wrong.")
```

Why bad: A bare `except:` catches *every* error, including ones you did not anticipate. This hides bugs. **Fix:** always catch specific exceptions like `except ValueError:` or `except FileNotFoundError:`.

13.8 File handling — opening files

To work with a file, you first **open** it. The `open()` function takes a file path and a *mode* that tells Python what you want to do with the file.

Mode	Meaning	Behavior
"r"	Read (default)	File must exist; error if not
"w"	Write	Creates file if it doesn't exist; overwrites if it does
"a"	Append	Creates file if it doesn't exist; adds to end if it does
"r+"	Read and write	File must exist

13.9 Reading from a text file

```
# Method 1: read the entire file as one string
with open("notes.txt", "r") as f:
    content = f.read()
print(content)

# Method 2: read line by line (best for large files)
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())    # .strip() removes trailing newline

# Method 3: read all lines into a list
with open("notes.txt", "r") as f:
    lines = f.readlines()
print(len(lines))    # number of lines
```

13.10 Writing to a text file

```
# Overwrite mode
with open("greeting.txt", "w") as f:
    f.write("Hello, World!\n")
    f.write("This is line 2.\n")

# Append mode
with open("greeting.txt", "a") as f:
    f.write("This line was added later.\n")
```

— REMEMBER — ALWAYS USE WITH FOR FILES

The `with` statement automatically closes the file when the block ends, even if an error occurs. Without it, you must remember to call `f.close()` yourself — and if you forget, the file may stay locked or data may not be saved. **Always use** `with open(...) as f:` for file operations.

13.11 CSV files

CSV (Comma-Separated Values) is a common format for tabular data — think of it as a spreadsheet saved as plain text. Python's `csv` module makes reading and writing CSV files easy.

```
import csv

# Writing to a CSV file
with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "roll", "grade"]) # header
    writer.writerow(["Aisha", "CSE001", "A"])
    writer.writerow(["Ravi", "CSE002", "B"])

# Reading from a CSV file
with open("students.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row) # each row is a list of strings
```

OUTPUT

```
['name', 'roll', 'grade']
['Aisha', 'CSE001', 'A']
['Ravi', 'CSE002', 'B']
```

Practice Questions

MULTIPLE CHOICE

Q1. Which module would you use to generate random numbers?

- A. `math`
- B. `random`
- C. `statistics`
- D. `os`

Q2. Which file mode creates a new file but does NOT overwrite an existing one?

- A. `"r"`
- B. `"w"`
- C. `"a"`
- D. `"x"`

Q3. What error is raised when you try to open a non-existent file in read mode?

- A. `ValueError`
- B. `FileNotFoundError`
- C. `TypeError`
- D. `IOError`

TRUE / FALSE

- Q4. The `with` statement automatically closes the file when the block ends.
- Q5. A bare `except:` (with no exception type) is the recommended way to handle errors.

OUTPUT PREDICTION

Q6. What is printed?

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print(result)
finally:
    print("Done")
```

SHORT CODING

- Q7. Write a program that asks the user for a number and prints its square root, using `math.sqrt()`. Handle the case where the user enters a negative number (which would cause a `ValueError`).
- Q8. Write a program that asks the user for 3 lines of text and writes them to a file called `notes.txt`. Then read the file back and print its contents to confirm.

— INTERVIEW CORNER

- Q1. What is the difference between `import math` and `from math import sqrt`?
- Q2. Why is `with open(...) as f:` preferred over `f = open(...)`?
- Q3. What is the difference between `"w"` mode and `"a"` mode when opening a file?

— FUN FACT

Python's standard library contains over 200 built-in modules covering everything from sending emails (`smtplib`) to compressing files (`zipfile`) to parsing HTML (`html.parser`) to building web servers (`http.server`). The Python community calls this the **"batteries included"** philosophy — Python comes with most of what you need out of the box, so you can build real programs without installing third-party packages.

QUICK RECAP

- A module is a Python file whose code can be imported by other files.
- `import module`, `from module import name`, `import module as alias`.
- Useful standard modules: `math`, `random`, `datetime`, `os`, `csv`, `json`, `statistics`.
- `try/except` catches errors so your program does not crash.
- Common exceptions: `ValueError`, `TypeError`, `ZeroDivisionError`, `FileNotFoundError`, `KeyError`, `IndexError`.
- File modes: `"r"` read, `"w"` overwrite, `"a"` append.
- Always use `with open(...) as f:` — it auto-closes the file.
- `csv.writer()` and `csv.reader()` handle CSV files.

HOMEWORK

1. Write a program that asks the user for their name and age, then writes this data to a file called `user.txt`. Read the file back and print its contents.
2. Write a robust number-input function `safe_int_input(prompt)` that keeps asking until the user enters a valid integer, then returns it. Use `try/except ValueError`.
3. Write a program that maintains a to-do list in a file. Each line is one task. Allow the user to add a task or view all tasks.
4. Write a program that reads a CSV file containing student names and marks, and prints the average mark.
5. Combine modules: write a program that uses `random` to pick a random number, `datetime` to timestamp the guess, and writes each guess to a log file using `with open(..., "a")`.

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (b) `random`
- Q2. (c) `"a"` (append)
- Q3. (b) `FileNotFoundError`
- Q4. True
- Q5. False — always catch specific exceptions
- Q6. `Cannot divide by zero` then `Done` (else skipped because exception occurred)

Expense Tracker Application

Chapter Project 4

A File-Backed Expense Tracker

Functions, modules, exceptions and CSV file handling — all working together in one practical app.

In this project you will build a small but real expense tracker. The user can add an expense (with date, category, amount, and a note), view all expenses, see the total spent, and see a category-wise summary. All data is saved to a CSV file, so it persists between runs. This is your first program that combines functions, file handling, and exception handling into a single coherent application.

Project introduction

Expense trackers are one of the most common beginner projects because they are genuinely useful. After this project, you will have a tool you can actually use to track your daily spending. More importantly, you will have practiced the full set of skills needed for any data-management application: structured input, file-backed storage, summary computation, and graceful error handling.

Objective

Build a Python program that:

1. Lets the user add an expense with a date, category, amount, and optional note.
2. Saves each expense to a CSV file (`expenses.csv`).
3. Lets the user view all expenses in a formatted table.
4. Shows the total amount spent.
5. Shows a per-category breakdown.
6. Handles file errors and invalid input gracefully.

Concepts used

CONCEPTS

Functions

CONCEPTS

Modules: csv, datetime

CONCEPTS

Exception handling

CONCEPTS

File handling (CSV)

CONCEPTS

Dictionaries (summary)

CONCEPTS

Menu-driven CLI

Algorithm — the logical approach

STEP-BY-STEP PLAN

- 1 Define functions for each action: `add_expense()` , `view_expenses()` , `total_expenses()` , `summary_by_category()` .
- 2 Use `datetime.date.today().isoformat()` to auto-fill the date when adding an expense.
- 3 Use `csv.writer` in append mode to add a new row for each expense.
- 4 Use `csv.reader` to load all expenses for view, total, and summary.
- 5 For the summary, use a dictionary mapping each category to its total amount.
- 6 Wrap file operations in `try/except` to handle missing or corrupt files.
- 7 Build a menu loop that calls the appropriate function based on user choice.

The complete Python code

```
# expense_tracker.py - a file-backed expense tracker
import csv
import datetime as dt
import os

EXPENSES_FILE = "expenses.csv"

def add_expense():
    """Prompt the user for an expense and append it to the CSV file."""
    date = input(" Date (YYYY-MM-DD, blank for today): ").strip()
    if not date:
        date = dt.date.today().isoformat()
    category = input(" Category (e.g. Food, Travel): ").strip()
    try:
        amount = float(input(" Amount in Rs: "))
    except ValueError:
        print(" Invalid amount. Expense not added.")
        return
    note = input(" Note (optional): ").strip()

    with open(EXPENSES_FILE, "a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([date, category, amount, note])
    print(f" Added: Rs.{amount:.2f} on {category} ({date}).")

def load_expenses():
    """Load all expenses from the CSV file. Returns a list of dicts."""
    if not os.path.exists(EXPENSES_FILE):
        return []
    expenses = []
    try:
        with open(EXPENSES_FILE, "r") as f:
            reader = csv.reader(f)
            for row in reader:
                if len(row) == 4:
                    expenses.append({
                        "date": row[0],
                        "category": row[1],
                        "amount": float(row[2]),
                        "note": row[3]
                    })
    except (FileNotFoundError, ValueError):
        print(" Warning: could not read expenses file. Starting fresh.")
        return []
    return expenses

def view_expenses():
    """Print all expenses in a formatted table."""
    expenses = load_expenses()
    if not expenses:
        print(" No expenses recorded yet.")
```

```

        return

    print(f"\n  {'Date':<12} {'Category':<12} {'Amount':>10}  Note")
    print("    " + "-" * 50)
    for e in expenses:
        print(f"      {e['date']:<12} {e['category']:<12} Rs.
{e['amount']:>7.2f}  {e['note']}"")

def total_expenses():
    """Print the total amount spent across all expenses."""
    expenses = load_expenses()
    total = sum(e["amount"] for e in expenses)
    print(f"  Total spent: Rs.{total:.2f} across {len(expenses)}
expenses.")

def summary_by_category():
    """Print a per-category breakdown of spending."""
    expenses = load_expenses()
    if not expenses:
        print("  No expenses to summarize.")
        return
    summary = {}
    for e in expenses:
        cat = e["category"]
        summary[cat] = summary.get(cat, 0) + e["amount"]
    print("\n  --- Summary by Category ---")
    for cat, amt in sorted(summary.items()):
        print(f"    {cat:<15} Rs.{amt:>8.2f}")

# Main menu loop
while True:
    print("\n≡≡≡ Expense Tracker ≡≡≡")
    print("1. Add expense")
    print("2. View all expenses")
    print("3. Show total")
    print("4. Summary by category")
    print("5. Exit")

    choice = input("Choose (1-5): ")
    if choice == "1":
        add_expense()
    elif choice == "2":
        view_expenses()
    elif choice == "3":
        total_expenses()
    elif choice == "4":
        summary_by_category()
    elif choice == "5":
        print("Goodbye!")
        break
    else:
        print("  Invalid choice.")

```

Code explanation

Imports and constant (lines 2–5): We import three modules — `csv` for CSV file handling, `datetime` for date operations, and `os` to check whether the expenses file exists. The filename is stored in a constant at the top so it is easy to change.

`add_expense()` **(lines 7–22):** Asks for the date (with a sensible default of today if blank), category, amount, and note. The amount conversion is wrapped in `try/except ValueError` so the program does not crash if the user types "abc" instead of a number. The expense is appended to the CSV file using `csv.writer` in append mode.

`load_expenses()` **(lines 24–45):** This is the workhorse — all the other functions call it. It checks whether the file exists, opens it, reads each row, and builds a list of dictionaries. The `try/except` block handles both missing files and corrupt rows (where the amount cannot be parsed as a float). The returned list is used by every other function.

`view_expenses()` **(lines 47–56):** Calls `load_expenses()` and prints the result in a formatted table. The `{value:<12}` syntax in f-strings left-aligns the value in a 12-character field; `{value:>10}` right-aligns in a 10-character field. This gives the table its neat columns.

`total_expenses()` **(lines 58–61):** Uses the built-in `sum()` function with a *generator expression* (`e["amount"] for e in expenses`) to add up all amounts. This is a clean, Pythonic way to total a list.

`summary_by_category()` **(lines 63–75):** Builds a dictionary mapping each category to the total amount spent on it. The pattern `summary.get(cat, 0) + amount` means "get the current total for this category (or 0 if not seen yet) and add the new amount". The result is then sorted and printed.

Main menu loop (lines 78–93): Same pattern as the Student Management project — a `while True` loop with `if/elif/else` dispatching to the appropriate function.

Sample session

SAMPLE SESSION (RUN TWICE TO SHOW PERSISTENCE)

≡ Expense Tracker ≡

1. Add expense

2. View all expenses

3. Show total

4. Summary by category

5. Exit

> Choose (1-5): 1

Date (YYYY-MM-DD, blank for today): (enter)

Category (e.g. Food, Travel): Food

Amount in Rs: 250

Note (optional): Lunch

Added: Rs.250.00 on Food (2025-08-08).

> Choose (1-5): 1

... (added Travel / Rs.80 / Auto)

> Choose (1-5): 1

... (added Food / Rs.120 / Tea & snacks)

> Choose (1-5): 4

--- Summary by Category ---

Food Rs. 370.00

Travel Rs. 80.00

> Choose (1-5): 3

Total spent: Rs.450.00 across 3 expenses.

> Choose (1-5): 5

Goodbye!

[Run the program again later – the expenses are still there!]

Possible improvements

- **Date filtering:** Add a menu option to view expenses for a specific month or date range.
- **Edit and delete:** Allow the user to remove or modify an existing expense.
- **Monthly summary:** Group expenses by month as well as by category.
- **Budget alerts:** Let the user set a monthly budget and warn when spending approaches the limit.
- **Export to JSON:** Add an option to export expenses to a JSON file (using the `json` module).

— PRACTICE CHALLENGE

Add a "5. Delete expense" menu option (renumber Exit to 6). It should display all expenses with an index number next to each, ask the user which index to delete, then rewrite the CSV file without that row.

— PROJECT REFLECTION

This project combines four chapters of learning into a single coherent application. The functions make the code modular and easy to read. The CSV module handles the file storage. Exception handling keeps the program robust against bad input and missing files. And the menu loop ties it all together into an interactive experience. This is what real Python applications look like — not one long script, but a set of focused functions, each doing one job well.

CHAPTER 15

Introduction to Object-Oriented Programming

So far you have organised your programs using functions — blocks of code that do one task. As programs get larger, you need a way to group related data and the functions that work on that data together. **Object-Oriented Programming (OOP)** is that way. This chapter introduces the core ideas of OOP — classes and objects — using simple real-life analogies, with no advanced theory beyond what Week 1 requires.

15.1 Why Object-Oriented Programming?

Imagine you are building a school management system. You have data about students (name, age, marks) and operations on students (calculate grade, promote, suspend). In a purely function-based program, you would pass student dictionaries around to separate functions: `calculate_grade(student)`, `promote(student)`, and so on. This works, but it scatters related logic across the program.

OOP offers a different approach: bundle the data and the operations together into a single unit called a **class**. A `Student` class would contain both the student's data *and* the methods that work on it. The class becomes a self-contained blueprint — you create individual student *objects* from it. This organisation scales much better as programs grow.

15.2 Classes and objects — the blueprint analogy

Think of a **class** as a blueprint for a car. The blueprint describes what a car has (wheels, engine, colour) and what it can do (drive, brake, honk). But the blueprint itself is not a car — you cannot drive a piece of paper. An **object** is an actual car built from that blueprint. You can build many cars (objects) from the same blueprint (class), each with its own colour and mileage but all sharing the same structure.

CLASS (BLUEPRINT)

```
class Car:
    wheels = 4
    def drive(): ...
    def honk(): ...
```

OBJECTS (INSTANCES)

```
my_car = Car("red")
your_car = Car("blue")
taxi = Car("yellow")
```

FIGURE 15.1 — A CLASS IS A BLUEPRINT; OBJECTS ARE INSTANCES BUILT FROM IT

15.3 Procedural vs Object-Oriented — a comparison

Aspect	Procedural (functions only)	Object-Oriented (classes)
Organisation	Data and functions are separate	Data and functions bundled together
Example	<pre>student = {"name": "A"}; promote(student)</pre>	<pre>student = Student("A"); student.promote()</pre>
Best for	Small scripts, simple utilities	Large applications, modelling real-world entities
Reusability	Functions can be reused	Classes can be reused and extended (in Week 2)

15.4 Defining a class in Python

Here is the simplest possible class — a `Dog` with a name and a method to make it bark.

```
class Dog:
    def __init__(self, name):
        self.name = name        # instance variable

    def bark(self):
        print(f"{self.name} says Woof!")
```

Let us unpack this carefully, because there are several new ideas here.

`class Dog:` The `class` keyword defines a new class. By convention, class names use CapitalisedWords (PascalCase).

`def __init__(self, name):` This is a special method called the **constructor**. Python calls it automatically whenever you create a new `Dog` object. The double underscores on both sides (called "dunder") tell Python this method has a special meaning.

`self` — This is the most confusing part for beginners. `self` refers to the specific object being created or operated on. When you write `self.name = name`, you are saying "set this particular dog's name to the value passed in". Every method in a class takes `self` as its first parameter — it is how Python knows which object the method is acting on.

`def bark(self):` A regular method. Notice it also takes `self` as the first parameter. Inside, `self.name` refers to *this particular dog's* name.

15.5 Creating objects

```
# Create two Dog objects
dog1 = Dog("Rex")
dog2 = Dog("Bella")

# Each has its own name
print(dog1.name)    # Rex
print(dog2.name)    # Bella

# Call the bark method on each
dog1.bark()         # Rex says Woof!
dog2.bark()         # Bella says Woof!
```

Notice we do *not* pass anything for `self` — Python fills it in automatically. When you write `dog1.bark()`, Python internally calls `Dog.bark(dog1)`, passing `dog1` as `self`. This is just a convenience; conceptually, the method is operating on a specific object.

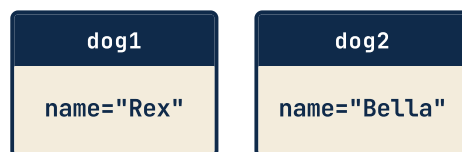


FIGURE 15.2 — TWO OBJECTS OF THE SAME CLASS, EACH WITH ITS OWN DATA

15.6 A complete example — Student class

Here is a slightly bigger example that brings together everything you have learned about classes. The `Student` class has three instance variables (name, roll, marks) and two methods (a method to get the grade, and a method to display the student's info).

```

class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks

    def get_grade(self):
        if self.marks ≥ 90:
            return "A"
        elif self.marks ≥ 80:
            return "B"
        elif self.marks ≥ 70:
            return "C"
        elif self.marks ≥ 60:
            return "D"
        else:
            return "F"

    def display(self):
        print(f"{self.roll} | {self.name} | Marks: {self.marks} | Grade: {self.get_grade()}")

# Create a few students
s1 = Student("Aisha", "CSE001", 85)
s2 = Student("Ravi", "CSE002", 72)
s3 = Student("Priya", "CSE003", 95)

s1.display()    # CSE001 | Aisha | Marks: 85 | Grade: B
s2.display()    # CSE002 | Ravi | Marks: 72 | Grade: C
s3.display()    # CSE003 | Priya | Marks: 95 | Grade: A

# Access instance variables directly
print(s1.name)    # Aisha
print(s3.get_grade())    # A

```

Notice how the `get_grade()` method uses `self.marks` — the marks of *whichever student the method was called on*. When you call `s1.get_grade()`, `self` is `s1`, so it uses `s1.marks` (85). When you call `s3.get_grade()`, `self` is `s3`, so it uses `s3.marks` (95). This is the heart of OOP — the same method definition works correctly for every object of the class.

15.7 Why this matters

By bundling data and behaviour together, classes give you three concrete benefits:

- **Code organisation:** All student-related code lives in one place. You do not hunt through dozens of functions to find what touches student data.
- **Modelling the real world:** A `Student` class mirrors how you actually think about students. The code reads more naturally.

- **Reusability:** Once you have a `Student` class, you can create as many students as you want with one line each. Adding a fourth student is trivial: `s4 = Student("Sam", "CSE004", 68)`.

— REMEMBER

- A **class** is a blueprint; an **object** is an instance built from it.
- `__init__` is the constructor — Python calls it automatically when you create an object.
- `self` refers to the current object. Every method takes `self` as its first parameter.
- Instance variables are attached to `self`: `self.name = name`.
- Call methods with `object.method()` — Python fills in `self` automatically.
- Class names use PascalCase: `Student`, `LibraryBook`, `ExpenseTracker`.

— COMMON MISTAKES

- **Forgetting `self` in method definitions:** `def bark():` → `TypeError`. Always use `def bark(self):`.
- **Forgetting one underscore on each side:** `_init_` or `__init` will not be called by Python. It must be exactly `__init__` (double underscore on both sides).
- **Forgetting parentheses when creating an object:** `s1 = Student` just aliases the class. You need `s1 = Student("Aisha", ...)`.
- **Accessing an instance variable before it is set:** if you forget to assign `self.name = name` in `__init__`, then `s1.name` raises `AttributeError`.

Practice Questions

MULTIPLE CHOICE

Q1. What is the purpose of the `__init__` method?

- A. It is called when an object is deleted.
- B. It is the constructor — called automatically when an object is created.
- C. It is an alternative name for the `main` function.
- D. It is used to import modules.

Q2. What does `self` refer to in a method?

- A. The class itself.
- B. The parent class.
- C. The specific object on which the method was called.
- D. A reserved Python keyword with no meaning.

OUTPUT PREDICTION

Q3. What is printed?

```
class Counter:
    def __init__(self):
        self.count = 0
    def inc(self):
        self.count += 1

c = Counter()
c.inc()
c.inc()
c.inc()
print(c.count)
```

TRUE / FALSE

Q4. A class can have multiple objects, each with its own instance variables.

Q5. You must always pass `self` explicitly when calling a method: `obj.method(self)`.

SHORT CODING

Q6. Write a `Book` class with instance variables `title` and `author`, and a method `display()` that prints `"Title by Author"`. Create two book objects and call `display()` on each.

Q7. Write a `Rectangle` class with `width` and `height` instance variables, and methods `area()` and `perimeter()` that return the corresponding values. Test with a 4×5 rectangle.

— INTERVIEW CORNER

Q1. What is the difference between a class and an object?

Q2. What is `__init__` and when is it called?

Q3. What is the purpose of `self`? Can you rename it to something else?

— FUN FACT

In Python, *everything* is an object — even integers, strings, and functions. When you write `x = 5`, `x` is an instance of the `int` class. When you write `"hello".upper()`, you are calling a method on an instance of the `str` class. This unified design — everything is an object — is one of Python's most elegant features, and it is why methods like `.upper()` work consistently across types.

QUICK RECAP

- A **class** bundles data (instance variables) and behaviour (methods) into one unit.
- An **object** is an instance of a class — built from the blueprint.
- `__init__` is the constructor, called automatically when an object is created.
- `self` refers to the current object; it is the first parameter of every method.
- Instance variables are stored on `self`: `self.name = name`.
- Create objects with `obj = ClassName(args)`.
- Call methods with `obj.method()` — Python passes `self` automatically.

HOMEWORK

1. Write a `BankAccount` class with instance variables `owner` and `balance`, and methods `deposit(amount)`, `withdraw(amount)`, and `display()`. Test by creating an account, depositing 1000, withdrawing 300, and displaying the result.
2. Write a `Temperature` class that stores a temperature in Celsius. Add methods `to_fahrenheit()` and `to_kelvin()` that return the converted values.
3. Write a `Product` class with `name`, `price`, and `quantity`. Add a method `total_value()` that returns `price * quantity`. Create 3 products and print the total value of each.
4. Write a `Circle` class with a `radius` instance variable. Add methods `area()` and `circumference()`. Use `math.pi` for π .
5. Refactor the Student Management Program (Project 3) to use a `Student` class instead of dictionaries. The list should now hold `Student` objects, and the menu should call methods on those objects.

ANSWER KEY — PRACTICE QUESTIONS

- Q1. (b) The constructor — called automatically when an object is created.
- Q2. (c) The specific object on which the method was called.
- Q3. `3` — count starts at 0, incremented 3 times.
- Q4. True
- Q5. False — Python passes `self` automatically; you call `obj.method()`.

Library Management System

Capstone Mini Project • Project 5

A Library Management System in Python

Bringing together everything you have learned this week — variables, control flow, data structures, functions, modules, files, exceptions, and OOP.

This is the capstone project for Week 1. You will build a small but functional Library Management System where users can add books, view all books, search by title or author, borrow a book, and return a book. All book data persists in a CSV file, so the library survives between runs. The system uses two classes — `Book` and `Library` — demonstrating how OOP helps organise a larger program.

Project introduction

A library management system is a classic project because it touches every concept in this handbook. Books need to be stored (lists, classes). Users need to interact (input, output, menu loop). Books need to be searched (loops, conditionals, string methods). State needs to persist (file handling, CSV). Things can go wrong (exceptions). And the whole program needs to be organised (functions, modules, classes). If you can build this, you can build almost any small Python application.

Objective

Build a Python program that:

1. Uses a `Book` class to represent individual books.
2. Uses a `Library` class to manage the collection of books.
3. Persists all book data to a CSV file (`library.csv`).
4. Loads the data when the program starts.
5. Provides a menu: Add book, View all, Search by title, Search by author, Borrow, Return, Exit.
6. Handles errors (file not found, invalid input, borrowing an already-borrowed book).

Concepts used — the full checklist

CONCEPTS

Variables & types

CONCEPTS

Operators

CONCEPTS

if/elif/else

CONCEPTS

for & while loops

CONCEPTS

Strings & methods

CONCEPTS

Lists & dicts

CONCEPTS

Functions

CONCEPTS

Modules: csv, os

CONCEPTS

Exception handling

CONCEPTS

File handling

CONCEPTS

Classes & objects

CONCEPTS

`__init__` & `self`

Algorithm — the design

STEP-BY-STEP PLAN

- 1 Define a `Book` class with title, author, and `is_borrowed` (a boolean). Methods: `borrow()`, `return_book()`, `display()`.
- 2 Define a `Library` class that holds a list of `Book` objects. Methods: `add_book()`, `view_all()`, `search_by_title()`, `search_by_author()`, `borrow_book()`, `return_book()`, `save_to_file()`, `load_from_file()`.
- 3 On startup, create a `Library` and call `load_from_file()`.
- 4 Show a menu and dispatch to the appropriate library method.
- 5 Whenever the library changes (add/borrow/return), call `save_to_file()` so data persists.

The complete Python code

```
# library_management.py - a small library management system
import csv
import os

LIBRARY_FILE = "library.csv"

class Book:
    """A single book in the library."""
    def __init__(self, title, author, is_borrowed=False):
        self.title = title
        self.author = author
        self.is_borrowed = is_borrowed

    def borrow(self):
        if self.is_borrowed:
            print(f" '{self.title}' is already borrowed.")
            return False
        self.is_borrowed = True
        print(f" You borrowed '{self.title}'.")
        return True

    def return_book(self):
        if not self.is_borrowed:
            print(f" '{self.title}' was not borrowed.")
            return False
        self.is_borrowed = False
        print(f" You returned '{self.title}'. Thank you!")
        return True

    def display(self):
        status = "[Borrowed]" if self.is_borrowed else "[Available]"
        print(f" {status} '{self.title}' by {self.author}")

class Library:
    """A collection of books, persisted to a CSV file."""
    def __init__(self):
        self.books = []
        self.load_from_file()

    def add_book(self, title, author):
        self.books.append(Book(title, author))
        print(f" Added '{title}' by {author}.")
        self.save_to_file()

    def view_all(self):
        if not self.books:
            print(" Library is empty.")
            return
        print("\n --- All Books ---")
        for book in self.books:
```

```

        book.display()

    def search_by_title(self, query):
        matches = [b for b in self.books if query.lower() in
b.title.lower()]
        if not matches:
            print(f" No books matching '{query}'.")
        else:
            for b in matches:
                b.display()

    def search_by_author(self, query):
        matches = [b for b in self.books if query.lower() in
b.author.lower()]
        if not matches:
            print(f" No books by author '{query}'.")
        else:
            for b in matches:
                b.display()

    def borrow_book(self, title):
        for b in self.books:
            if b.title.lower() == title.lower():
                if b.borrow():
                    self.save_to_file()
                return
        print(f" No book titled '{title}' found.")

    def return_book(self, title):
        for b in self.books:
            if b.title.lower() == title.lower():
                if b.return_book():
                    self.save_to_file()
                return
        print(f" No book titled '{title}' found.")

    def save_to_file(self):
        with open(LIBRARY_FILE, "w", newline="") as f:
            writer = csv.writer(f)
            for b in self.books:
                writer.writerow([b.title, b.author, "yes" if b.is_borrowed
else "no"])

    def load_from_file(self):
        if not os.path.exists(LIBRARY_FILE):
            return
        try:
            with open(LIBRARY_FILE, "r") as f:
                reader = csv.reader(f)
                for row in reader:
                    if len(row) == 3:
                        title, author, borrowed = row
                        self.books.append(Book(title, author, borrowed ==
"yes"))

```

```

        except (FileNotFoundError, ValueError):
            print(" Warning: could not load library file. Starting
empty.")

# Main program
def main():
    lib = Library()
    while True:
        print("\n=== Library Management System ===")
        print("1. Add book")
        print("2. View all books")
        print("3. Search by title")
        print("4. Search by author")
        print("5. Borrow a book")
        print("6. Return a book")
        print("7. Exit")
        choice = input("Choose (1-7): ")

        if choice == "1":
            title = input(" Title: ")
            author = input(" Author: ")
            lib.add_book(title, author)
        elif choice == "2":
            lib.view_all()
        elif choice == "3":
            lib.search_by_title(input(" Title to search: "))
        elif choice == "4":
            lib.search_by_author(input(" Author to search: "))
        elif choice == "5":
            lib.borrow_book(input(" Title to borrow: "))
        elif choice == "6":
            lib.return_book(input(" Title to return: "))
        elif choice == "7":
            print("Goodbye!")
            break
        else:
            print(" Invalid choice.")

if __name__ == "__main__":
    main()

```

Code explanation — organised by class

The **Book class** models a single book. Each book has a title, an author, and a borrowed status. The `borrow()` and `return_book()` methods encapsulate the rules: you cannot borrow an already-borrowed book, and you cannot return a book that was not borrowed. The `display()` method shows the book's status. Notice how the rules live *inside* the class — not scattered across the main program. This is the OOP advantage.

The **Library class** manages the collection. Its `__init__` creates an empty list and immediately loads any existing data from the CSV file — so the library "remembers" its state

between runs. Every method that modifies the books list (add, borrow, return) calls `save_to_file()` so the change persists. The search methods use list comprehensions — a compact Python feature — to filter books matching the query, case-insensitively.

The `main()` function ties everything together. It creates a `Library` object, then runs the standard menu loop. Each menu option delegates to a library method — the menu itself contains almost no business logic. This separation of concerns (the menu handles user interaction; the library handles book logic) is the hallmark of well-structured code.

The `if __name__ == "__main__":` line at the bottom is a Python idiom. It means "only run `main()` if this file is being run directly, not if it is being imported as a module." This lets you (or other people) import this file and reuse the `Library` class without triggering the menu. It is a small but important professional habit.

Sample session

SAMPLE SESSION

```
≡≡≡ Library Management System ≡≡≡
1. Add book
2. View all books
3. Search by title
4. Search by author
5. Borrow a book
6. Return a book
7. Exit
> Choose (1-7): 1
Title: The Alchemist
Author: Paulo Coelho
Added 'The Alchemist' by Paulo Coelho.

> Choose (1-7): 1
Title: Wings of Fire
Author: A.P.J. Abdul Kalam
Added 'Wings of Fire' by A.P.J. Abdul Kalam.

> Choose (1-7): 2
--- All Books ---
[Available] 'The Alchemist' by Paulo Coelho
[Available] 'Wings of Fire' by A.P.J. Abdul Kalam

> Choose (1-7): 5
Title to borrow: The Alchemist
You borrowed 'The Alchemist'.

> Choose (1-7): 2
--- All Books ---
[Borrowed] 'The Alchemist' by Paulo Coelho
[Available] 'Wings of Fire' by A.P.J. Abdul Kalam

> Choose (1-7): 7
Goodbye!
```

[Run again later – the books and their borrowed status persist!]

Possible improvements

- **Member management:** Track who borrowed which book. Add a `Member` class.
- **Due dates:** Add a due date when borrowing, and compute fines for overdue returns.
- **Multiple copies:** Allow multiple copies of the same book, tracked by quantity.
- **Export to JSON:** Add a menu option to export the library to a JSON file (using the `json` module).
- **Search by genre:** Add a genre field to each book and a menu option to search by it.

— PRACTICE CHALLENGE

Add a "Display all borrowed books" menu option that prints only the books currently borrowed. (Hint: use a list comprehension to filter `self.books` for books where `b.is_borrowed` is True.)

— PROJECT REFLECTION — WEEK 1 CLOSING THOUGHT

Look at the code for this project. In about 110 lines, you have used every major concept from this week — variables, types, operators, control flow, loops, strings, lists, dictionaries, functions, modules, exception handling, file I/O, and OOP. A week ago, this code would have looked like alien script. Today, you can read it, understand it, and modify it. That is what learning to program looks like — not a single dramatic leap, but the steady accumulation of small skills, each one building on the last, until one day you look back and realise you can build things you could not build before. Congratulations on completing Week 1.

CHAPTER 17

Weekly Revision, Quiz & What's Next

You have reached the end of Week 1. This final chapter does three things: it gives you a concise revision of every concept covered, it tests your understanding with a balanced 15-question quiz, and it gives you a glimpse of what Week 2 will bring. Take your time with the quiz — the goal is not to score perfectly but to identify which concepts you should revisit.

17.1 Week 1 revision notes

Below is a one-line summary of each chapter. Use this as a quick reference and as a checklist — if any line feels unfamiliar, go back and re-read that chapter.

Chapter	Topic	One-line summary
01	Orientation	Python is a readable, beginner-friendly language used in web, data science, AI and research.
02	Environment setup	Install Python, VS Code and Google Colab; verify with <code>python --version</code> .
03	Variables & types	Variables hold values; basic types are <code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code> , <code>None</code> .
04	Operators	Arithmetic, comparison, logical, assignment operators with precedence rules.
05	Calculator project	Combined variables, input, output and operators into a working program.
06	Decisions	<code>if</code> , <code>elif</code> , <code>else</code> ; indentation marks blocks; colons end <code>if</code> lines.
07	Loops	<code>for</code> with <code>range()</code> ; <code>while</code> for condition-based; <code>break</code> and <code>continue</code> .
08	Guessing game	Combined decisions and loops with the <code>random</code> module.
09	Strings	Indexing, slicing, immutability, methods (<code>.upper()</code> , <code>.strip()</code> , <code>.split()</code> , ...).
10	Collections	Lists (mutable), tuples (immutable), sets (unique), dicts (key-value).
11	Student management	Menu-driven CLI app combining collections and control flow.
12	Functions	<code>def</code> , parameters, <code>return</code> , defaults, scope.
13	Modules, files, exceptions	<code>import</code> , <code>try/except</code> , file modes, CSV, <code>with</code> statement.
14	Expense tracker	Functions + CSV persistence + exceptions in one practical app.
15	OOP basics	Classes, objects, <code>__init__</code> , <code>self</code> , instance variables, methods.
16	Library system	Capstone combining all concepts with two classes and CSV persistence.

17.2 Skills you have gained this week

If you have worked through this handbook and attempted the projects, you can now honestly say the following about yourself. Read each line and tick it off mentally — this is your achievement list for Week 1.

— CONFIDENCE CHECKLIST

- I can install Python and set up VS Code / Google Colab on my own.
- I can write a Python program that takes input, processes it, and prints output.
- I understand variables, the five basic data types, and type conversion.
- I can use arithmetic, comparison, and logical operators correctly.
- I can write programs that make decisions using `if/elif/else`.
- I can repeat actions using `for` and `while` loops.
- I can manipulate text using string methods and slicing.
- I can choose between lists, tuples, sets, and dictionaries based on the problem.
- I can organise code into reusable functions with parameters and return values.
- I can read from and write to text and CSV files using `with`.
- I can handle errors gracefully with `try/except`.
- I can define a class, create objects, and write methods that use `self`.
- I can build a small menu-driven CLI application from scratch.
- I can debug simple errors by reading error messages and tracing through code.

If most of these feel true, you are well prepared for Week 2. If a few feel shaky, that is completely normal — go back to the relevant chapter, re-do the practice questions, and try the homework again. Programming skills deepen with repetition.

17.3 Weekly quiz — 15 questions

This quiz is balanced across five categories: conceptual understanding (Q1–Q5), logical reasoning (Q6–Q7), debugging (Q8–Q9), output prediction (Q10–Q12), and short coding (Q13–Q15). Attempt all questions before checking the answer key at the end of this chapter.

CONCEPTUAL UNDERSTANDING (MCQS)

Q1. Which Python data type is used to store text?

- A. `int`
- B. `str`
- C. `bool`
- D. `float`

Q2. What is the output of `7 // 2` in Python?

- A. 3.5
- B. 3
- C. 4
- D. 1

Q3. Which collection does NOT allow duplicate elements?

- A. List
- B. Tuple
- C. Set
- D. Dictionary

Q4. What does the `self` parameter in a class method refer to?

- A. The class itself
- B. The current object on which the method is called
- C. The parent class
- D. A reserved Python keyword

Q5. Which statement correctly opens a file for appending without overwriting existing content?

- A. `open("f.txt", "r")`
- B. `open("f.txt", "w")`
- C. `open("f.txt", "a")`
- D. `open("f.txt", "x")`

LOGICAL REASONING

Q6. You need to store the marks of 30 students and quickly look up any student's marks by their roll number. Which data structure is the best choice, and why?

Q7. A function `calculate_discount(price, rate=0.1)` is called as `calculate_discount(500)`. What is the value of `rate` inside the function? What if it is called as `calculate_discount(500, 0.2)` ?

DEBUGGING

Q8. The following code is supposed to print "Adult" for users aged 18+, but it always prints "Minor". Find and explain the bug:

```
age = input("Age: ")
if age ≥ 18:
    print("Adult")
else:
    print("Minor")
```

Q9. This loop is supposed to print numbers 1 to 5, but it prints 1, 2, 3, 4 and stops. Why?

```
for i in range(1, 5):
    print(i)
```

OUTPUT PREDICTION

Q10. What is printed?

```
s = "Python"
print(s[1:4], s[-2:])
```

Q11. What is printed?

```
def f(x, y=5):
    return x * y

print(f(3), f(3, 2))
```

Q12. What is printed?

```
d = {"a": 1, "b": 2, "c": 3}
print(sum(d.values()))
```

SHORT CODING

Q13. Write a function `is_palindrome(s)` that returns `True` if `s` reads the same forwards and backwards (case-insensitive). Test with `"Madam"` (should be `True`) and `"Python"` (should be `False`).

Q14. Write a `for` loop that prints all even numbers from 10 to 30 (inclusive), each on a new line.

Q15. Define a `Circle` class with a `radius` attribute and an `area()` method that returns `3.14159 * radius * radius`. Create a circle with radius 5 and print its area.

17.4 What's coming in Week 2

Week 1 gave you the foundations — you can now write small but complete Python programs. Week 2 builds on this foundation and takes you toward intermediate Python. Expect to dive deeper into object-oriented programming (inheritance, polymorphism, encapsulation), work with more file formats (JSON, maybe Excel), explore external libraries via `pip`, and probably build your first small project that interacts with the web or with a database. The exact topics will follow your course syllabus, but the trajectory is clear: from "I can write a script" to "I can build a small application that solves a real problem".

The single best thing you can do between now and Week 2 is to keep coding. Spend 15 minutes a day writing small Python programs — anything that catches your interest. A dice roller. A tip calculator. A program that asks for your mood and prints a song recommendation. The size of the program does not matter; what matters is that you keep your new skills warm. Programming is a muscle — it grows with use and atrophies with disuse.

17.5 Final words

If this is your first week of programming, you have just done something genuinely hard. You have learned a new way of thinking — precise, logical, patient. You have built five working programs with your own hands. You have made mistakes, fixed them, and made new mistakes. That is what programming is, and that is what programming always will be — even for people who have done it for thirty years. The mistakes just become more interesting.

Be patient with yourself. Be curious. Be willing to look things up. Read other people's code when you can find it. And above all, keep typing. The next line of code you write is more important than the last one you read about. Welcome to programming — you are one of us now.

ANSWER KEY — WEEKLY QUIZ

(Only MCQs and output-prediction questions have answers. Coding questions you verify by running them.)

- Q1. (b) `str`
- Q2. (b) 3 — floor division truncates the decimal
- Q3. (c) Set
- Q4. (b) The current object on which the method is called
- Q5. (c) `open("f.txt", "a")` — append mode
- Q6. A dictionary — roll number as key, marks as value. Lookup is $O(1)$ (instant) by key, which is exactly what we need.
- Q7. First call: `rate` = 0.1 (default). Second call: `rate` = 0.2 (explicitly passed).
- Q8. `input()` returns a string. Comparing a string to an integer with `>=` raises `TypeError` in Python 3 (or compares lexicographically in Python 2). Fix: `age = int(input("Age: "))`.
- Q9. `range(1, 5)` produces 1, 2, 3, 4 — the stop value 5 is exclusive. To include 5, use `range(1, 6)`.
- Q10. `yth on` — `s[1:4]` is `yth`, `s[-2:]` is `on`.
- Q11. `15 6` — first call uses default `y=5` ($3*5=15$), second uses `y=2` ($3*2=6$).
- Q12. `6` — sum of values $1+2+3$.
- Q13–15. Verify by running your code.



Keep Coding.

*You have completed Week 1 of Python Programming
Foundations. The journey has only just begun.*

END OF HANDBOOK • EDITION 1.0

